

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

VYUŽITIE UMELEJ INTELIGENCIE PRI VÝVOJI
SOFTVÉRU
BAKALÁRSKA PRÁCA

2026
SAMUEL SOKOL

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

VYUŽITIE UMELEJ INTELIGENCIE PRI VÝVOJI
SOFTVÉRU

BAKALÁRSKA PRÁCA

Študijný program: Aplikovaná informatika
Študijný odbor: Informatika
Školiace pracovisko: Katedra aplikovanej informatiky
Školiteľ: doc. Ing. Ivan Polášek, PhD.

Bratislava, 2026
Samuel Sokol



ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Samuel Sokol
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Využitie umelej inteligencie pri vývoji softvéru
Using artificial intelligence in software development

Anotácia: Rastúce požiadavky a zložitosť softvérových systémov si vyžadujú sofistikovaný a kreatívny proces analýzy a návrhu, ktorý zahŕňa intenzívnu spoluprácu mnohých odborníkov z rôznych špecializácií, ktorí sú až v záverečnej fáze procesu analýzy a vývoja informovaní o skutočnom stave a problémoch. Jednou z hlavných motivácií tohto výskumu je preto analýza a návrh rôznych vhodných metód umelej inteligencie v jednej z nasledujúcich oblastí výskumu: generovanie zdrojového kódu, porozumenie softvéru a dokumentácia.

Cieľ: Navrhnete agentický AI systém na generovanie zdrojového kódu z prirodzeného jazykového zadania, ktorý zahŕňa formalizáciu zadania do štruktúrovanej reprezentácie (prípad použitia), využíva princípy fragmentácie problému, rozdelenia úloh medzi špecializovaných agentov a iteratívneho zlepšovania riešenia.
Implementujte takýto systém vrátane jednotlivých agentov, mechanizmov validácie (syntaktickej, sémantickej a testovacej), využitia externej znalostnej bázy (GraphRAG) a spracovania spätnej väzby používateľa (human-in-the-loop).
Experimentálne overte navrhnutý prístup na vybranom probléme a analyzujte jeho prínosy a obmedzenia z hľadiska správnosti a kvality kódu v porovnaní s bežným generovaním pomocou veľkých jazykových modelov a existujúcim agentickým systémom.

Vedúci: doc. Ing. Ivan Polášek, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 11.04.2025

Dátum schválenia: 28.04.2026

doc. RNDr. Damas Gruska, PhD.
garant študijného programu

PodĎakovanie: Veľmi pekne ďakujem docentovi Ing. Ivanovi Poláškov, PhD. za odborné vedenie, cenné rady, pripomienky a čas venovaný pri tvorbe tejto bakalárskej práce. Zároveň ďakujem svojej rodine a blízkym za podporu počas štúdia.

Abstrakt

Táto bakalárska práca sa zaoberá využitím umelej inteligencie pri vývoji softvéru, konkrétne návrhom a implementáciou agentického systému na generovanie zdrojového kódu z prirodzeného jazykového zadania. Práca analyzuje možnosti využitia veľkých jazykových modelov, agentických systémov, GraphRAG prístupov a štruktúrovaných znalostí v oblasti UI-asistovaného softvérového inžinierstva.

Na základe vykonanej analýzy bol navrhnutý viacagentový systém implementovaný pomocou frameworku LangGraph. Navrhované riešenie využíva fragmentáciu problému do viacerých krokov, postupnú transformáciu zadania do štruktúrovanej reprezentácie prípadu použitia, GraphRAG znalostnú bázu realizovanú pomocou Neo4j a iteratívnu validáciu generovaných výstupov.

Systém bol experimentálne porovnaný s priamym generovaním pomocou modelu Mistral a s agentickým systémom Roo Code. Evaluácia ukázala, že navrhovaný agentický systém je schopný v niektorých prípadoch generovať plne funkčné riešenia už na prvý pokus a dosahovať vysokú kvalitu kódu. Zároveň sa však potvrdila vyššia miera nedeterministickosti v porovnaní s priamym generovaním pomocou jednej LLM, čo sa prejavilo aj na variabilite výsledkov.

Výsledky zároveň naznačili, že využitie GraphRAG znalostnej bázy prispieva k zlepšeniu stability a funkčnej správnosti generovaného kódu.

Kľúčové slová: veľké jazykové modely, agentické systémy, generovanie zdrojového kódu, GraphRAG, LangGraph, softvérové inžinierstvo, umelá inteligencia

Abstract

This bachelor's thesis focuses on the use of artificial intelligence in software development, specifically on the design and implementation of an agentic system for source code generation from natural language input. The thesis analyzes the possibilities of using large language models, agentic systems, GraphRAG approaches, and structured knowledge in the field of AI-assisted software engineering.

Based on the conducted analysis, a multi-agent system implemented using the LangGraph framework was designed. The proposed solution utilizes problem fragmentation into multiple steps, gradual transformation of the input task into a structured use case representation, a GraphRAG knowledge base implemented using Neo4j, and iterative validation of generated outputs.

The system was experimentally compared with direct generation using the Mistral model and with the Roo Code agentic system. The evaluation showed that the proposed agentic system is capable of generating fully functional solutions on the first attempt in some cases while achieving high code quality. At the same time, a higher degree of nondeterminism compared to direct generation using a single LLM was observed, which resulted in variability of the outcomes.

The results also indicate that the use of a GraphRAG knowledge base contributes to improved stability and functional correctness of the generated code.

Keywords: large language models, agentic systems, source code generation, GraphRAG, LangGraph, software engineering, artificial intelligence

Obsah

Úvod	1
1 Analýza	3
1.1 Generovanie zdrojového kódu pomocou veľkých jazykových modelov . .	3
1.1.1 Typy veľkých jazykových modelov	3
1.1.2 Silné a slabé stránky LLM pri generovaní kódu	4
1.1.3 Prístupy k zlepšeniu generovania kódu	5
1.2 Nástroje pre AI-asistovaný vývoj softvéru	6
1.2.1 Najpoužívannejšie nástroje pre AI-asistovaný vývoj	6
1.2.2 Analýza kvality kódu generovaného AI nástrojmi	7
1.3 GraphRAG a štruktúrované znalosti	8
1.3.1 Základný princíp	9
1.3.2 Prínosy GraphRAG	9
1.3.3 Ontológia a reprezentácia znalostí	9
1.3.4 Obmedzenia, riziká a praktické požiadavky	10
1.4 Prípady použitia	11
1.5 Vektorová reprezentácia	11
1.6 Agentické systémy	12
1.6.1 Od jazykových modelov k autonómnym agentom	12
1.6.2 Multi-agentové systémy v softvérovom inžinierstve	12
1.6.3 Výhody a obmedzenia agentických prístupov	13
1.7 Cieľ práce, výskumné otázky a hypotézy	14
1.7.1 Súvisiace práce	14
1.7.2 Cieľ práce	14
1.7.3 Výskumné otázky a hypotézy	15
2 Návrh riešenia	17
2.1 Výber jazykového modelu	17
2.2 Prehľad architektúry	18
2.3 Elektronická príloha implementácie	19
2.4 Agentický pracovný postup	20

2.4.1	Generovanie prípadu použitia	21
2.4.2	Návrh architektúry systému	24
2.4.3	Znalostná báza (GraphRAG)	25
2.4.4	Generovanie metód	29
2.4.5	Integrácia programu a validácia	32
2.4.6	Výstup systému a spracovanie spätnej väzby	34
3	Evaluácia	39
3.1	Testovacie subjekty	39
3.2	Testovacie zadania	40
3.3	Testovacia metodika	41
3.4	Hodnotiace metriky	42
3.5	Elektronická príloha evaluácie	44
3.6	Výsledky evaluácie	45
3.6.1	Rezervačný systém	45
3.6.2	E-shop backend	49
3.7	Diskusia výsledkov	55
3.7.1	Priame generovanie pomocou Mistral	55
3.7.2	Navrhovaný agentický systém bez znalostnej bázy	56
3.7.3	Agentický systém s GraphRAG	56
3.7.4	Agentický systém Roo Code	57
3.7.5	Súhrn diskusie	58
3.7.6	Vyhodnotenie výskumných otázok a hypotéz	60
	Záver	61
	Príloha A	69

Zoznam obrázkov

1.1	Miera akceptácie žiadostí o prijatie zmien (prevzaté z [14])	6
1.2	Porovnanie architektúr klasického RAG a GraphRAG. (prevzaté z [21])	10
1.3	Schéma agentického pracovného postupu ilustrujúca cyklus plánovania, vykonávania úloh, hodnotenia výsledkov a iteratívneho zlepšovania riešenia v autonómnych agentických systémoch (prevzaté z [5]).	13
2.1	Vysokoúrovňová architektúra navrhovaného agentického systému. . . .	18
2.2	Štruktúra implementačnej časti repozitára	20
2.3	Ukážka grafovej reprezentácie znalostnej bázy v Neo4j.	27
2.4	Ukážka používateľského rozhrania systému.	36
3.1	Štruktúra evaluačnej časti repozitára	44

Zoznam tabuliek

1.1	Porovnanie výkonu všeobecných a kódovo orientovaných jazykových modelov (prevzaté z [8]).	4
1.2	Typy chýb v kóde generovanom LLM vyjadrené percentuálne (prevzaté z [4]).	5
1.3	Porovnanie vlastností kódu generovaného nástrojmi ChatGPT a GitHub Copilot na základe analyzovaných štúdií.	8
3.1	Výsledky LLM Mistral – rezervačný systém	46
3.2	Výsledky agentického systému bez GraphRAG – rezervačný systém . .	47
3.3	Výsledky agentického systému s GraphRAG – rezervačný systém	47
3.4	Výsledky agentického systému Roo Code – rezervačný systém	48
3.5	Súhrn výsledkov – rezervačný systém	49
3.6	Výsledky LLM Mistral – e-shop backend	50
3.7	Výsledky agentického systému bez GraphRAG – e-shop backend	51
3.8	Výsledky agentického systému s GraphRAG – e-shop backend	52
3.9	Výsledky agentického systému Roo Code – e-shop backend	54
3.10	Súhrn výsledkov – e-shop backend	54
3.11	Súhrn výhod a nevýhod jednotlivých prístupov	59

Úvod

Rozvoj veľkých jazykových modelov a generatívnej umelej inteligencie v posledných rokoch výrazne ovplyvnil spôsob, akým vzniká softvér. Nástroje ako GitHub Copilot, ChatGPT, alebo Roo Code umožňujú automatické generovanie zdrojového kódu, návrhov architektúry, dokumentácie či testovacích scenárov priamo z prirodzeného jazyka. Umelá inteligencia sa tak postupne stáva aktívnym účastníkom procesu vývoja softvéru a mení spôsob práce softvérových inžinierov.

Napriek výraznému pokroku však generovanie zdrojového kódu pomocou veľkých jazykových modelov stále prináša viacero problémov. Modely často generujú syntakticky korektný, no funkčne nesprávny kód, nedokážu dostatočne pracovať s globálnym kontextom systému a pri komplexnejších úlohách vykazujú výraznú nestabilitu výsledkov. Súčasný výskum sa preto čoraz viac zameriava na agentické systémy, ktoré rozdeľujú riešenie problému do viacerých krokov, využívajú iteratívnu validáciu a kombinujú generovanie s externými znalosťami.

Jedným z perspektívnych smerov je prepojenie veľkých jazykových modelov so štruktúrovanými znalosťami pomocou prístupov typu Retrieval-Augmented Generation a GraphRAG. Využitie znalostných báz, grafových databáz a ontológií môže modelom pomôcť lepšie pracovať s kontextom, návrhovými pravidlami a implementačnými príkladmi. Zároveň sa rozvíjajú agentické frameworky umožňujúce koordináciu viacerých špecializovaných agentov pri riešení komplexných úloh softvérového inžinierstva.

Práca je rozdelená do viacerých kapitol:

- **Kapitola 1 – Analýza**

Kapitola sa venuje analýze UI-asistovaného softvérového inžinierstva, generovania zdrojového kódu pomocou veľkých jazykových modelov, agentických systémov, GraphRAG prístupov a reprezentácie znalostí pomocou ontológií. Súčasťou kapitoly je aj formulácia cieľov práce, výskumných otázok a hypotéz.

- **Kapitola 2 – Návrh riešenia**

Kapitola opisuje návrh navrhovaného agentického systému, jeho architektúru, pracovný postup jednotlivých agentov a návrh GraphRAG znalostnej bázy. Zároveň sa venuje implementácii jednotlivých komponentov systému.

- **Kapitola 3 – Evaluácia**

Kapitola obsahuje experimentálne testovanie navrhovaného systému a porovnanie výsledkov s priamym generovaním pomocou veľkého jazykového modelu a existujúcim agentickým systémom. Evaluácia je realizovaná pomocou metrík zameraných na funkčnú správnosť a kvalitu generovaného kódu.

- **Záver**

V závere práce sú zhrnuté dosiahnuté výsledky, hlavné prínosy práce a možné smery ďalšieho vývoja navrhovaného systému.

Motiváciou tejto práce bolo preskúmať možnosti využitia agentických systémov pri generovaní zdrojového kódu a overiť, či dokáže kombinácia fragmentácie problému, iteratívnej validácie a GraphRAG znalostnej bázy zlepšiť funkčnú správnosť a kvalitu generovaných riešení. Cieľom práce je navrhnúť a implementovať vlastný agentický systém schopný transformovať prirodzené jazykové zadanie do funkčnej implementácie programu a následne experimentálne porovnať jeho vlastnosti s priamym generovaním pomocou veľkého jazykového modelu a s existujúcim agentickým systémom.

Kapitola 1

Analýza

1.1 Generovanie zdrojového kódu pomocou veľkých jazykových modelov

V súčasnom softvérovom inžinierstve predstavujú veľké jazykové modely (large language models, LLM) dominantnú triedu systémov umelej inteligencie (UI) využívaných na automatizované generovanie zdrojového kódu. Vďaka tréningu na rozsiahlych korpusoch kombinujúcich prirodzený jazyk a zdrojový kód dokážu tieto modely zachytávať syntaktické vzory a opakujúce sa štrukturálne schémy programov [10]. Súčasný výskum ukazuje, že LLM možno efektívne využiť v širokom spektre úloh, od jednoduchého dopĺňania kódu až po generovanie celých funkcií, testov či častí aplikačnej logiky [10]. Tento potenciál je však zároveň sprevádzaný významnými obmedzeniami.

1.1.1 Typy veľkých jazykových modelov

Existujúce práce rozlišujú medzi dvoma základnými triedami LLM používaných na generovanie kódu. Prvú skupinu tvoria všeobecné jazykové modely, ktoré sú primárne trénované na prirodzenom jazyku a až sekundárne aplikované na programátorské úlohy. Tieto modely vynikajú schopnosťou porozumieť zadaniu, no často vykazujú nižšiu presnosť pri dodržiavaní striktnej syntaxe a sémantiky programovacích jazykov. Druhú skupinu tvoria modely špecializované na kód, ktoré sú trénované alebo doladené na rozsiahlych dátach obsahujúcich zdrojový kód. Tieto modely dosahujú vyššiu syntaktickú správnosť a lepšie zvládajú dlhodobé závislosti medzi premennými, funkciami a dátovými štruktúrami [10]. Empirické porovnanie výkonu všeobecných a kódovo orientovaných jazykových modelov (tabuľka 1.1) ukazuje, že zatiaľ čo všeobecné modely dosahujú konkurencieschopné výsledky na výkonnostných testoch s otvorenou doménou, pri doménovo špecifických datasetoch výrazne zaostávajú za modelmi trénovanými priamo na zdrojovom kóde [8].

Tabuľka 1.1: Porovnanie výkonu všeobecných a kódovo orientovaných jazykových modelov (prevzaté z [8]).

Dataset	General-purpose		Code-oriented			
	ChatGPT-3.5-154B		CodeLlama-7B		PolyCoder-2.7B	
	BLEU	CodeBLEU	BLEU	CodeBLEU	BLEU	CodeBLEU
<i>Open-domain datasets</i>						
HumanEval	39.00	30.05	18.13	17.99	22.22	13.81
CodeSearchNet (Golang)	11.16	25.29	16.14	19.63	15.97	19.56
<i>Domain-specific datasets</i>						
Gin	5.81	17.73	8.47	18.07	8.13	17.05
Prometheus	1.75	12.17	1.79	13.13	1.77	12.11
gRPC-go	2.53	12.15	57.61	60.39	55.36	57.52
Unreal Engine	5.22	10.57	0.73	10.21	0.94	9.87
Cocos2d-x	3.71	12.92	16.76	25.87	13.83	23.83
Bgfx	0.86	8.09	0.55	7.72	0.76	7.16

1.1.2 Silné a slabé stránky LLM pri generovaní kódu

LLM sú obzvlášť efektívne pri generovaní krátkeho a stredne zložitého kódu, kde je riešenie možné odvodiť z lokálnych vzorov a bežných programátorských konštrukcií. Táto schopnosť robí z LLM vhodný nástroj na rýchle prototypovanie a podporu vývojára pri rutinných úlohách [10].

Na druhej strane viaceré rozsiahle empirické analýzy ukazujú, že generovaný kód často obsahuje netriviálne chyby. Medzi najčastejšie patria logické chyby, nesprávne ošetrovanie okrajových prípadov, chybné používanie aplikačných rozhraní a nekonzistentná práca so stavom programu [23, 4]. To ilustruje tabuľka 1.2, z ktorej vyplýva, že dominantnú časť tvoria funkčné a logické chyby, zatiaľ čo syntaktické chyby sa vyskytujú len zriedkavo. Významným problémom je aj skutočnosť, že mnohé z týchto chýb nie sú zachytiteľné bežnými automatickými metrikami ani jednoduchými jednotkovými testami, čo vedie k nadhodnocovaniu kvality generovaného kódu na výkonnostných testoch [4]. Tie nazachytia ani správanie v reálnych projektoch, kde sú UI-generované príspevky výrazne menej často akceptované než ľudské [14]. Obr. 1.1 ilustruje rozdiely v miere akceptácie žiadostí o prijatie zmien.

Osobitnou výzvou je generovanie doménovo špecifického kódu. Štúdie zamerané na túto oblasť ukazujú, že modely bez doménovej adaptácie často zlyhávajú pri používaní špecifických knižníc, porušujú doménové pravidlá alebo generujú neexistujúce rozhrania. Prenos znalostí medzi rôznymi doménami je obmedzený a úspech v jednej oblasti neznamena automaticky dobrý výkon v inej. Tieto zistenia poukazujú na limity univerzálneho prístupu k generovaniu kódu a zdôrazňujú potrebu cielenej adaptácie modelov [8].

Ďalším významným aspektom je bezpečnosť. Bez explicitných inštrukcií majú LLM

Tabuľka 1.2: Typy chýb v kóde generovanom LLM vyjadrené percentuálne (prevzaté z [4]).

Bug types	HumanEval plus			MBPP plus			APPS plus		
	GPT-4	GPT-3.5	Claude-3	GPT-4	GPT-3.5	Claude-3	GPT-4	GPT-3.5	Claude-3
A.1 Incomplete syntax structure	0.0	0.6	0.0	0.0	0.5	0.0	0.2	1.5	0.2
A.2 Incorrect indentation	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.2	0.0
A.3 Library import error	0.6	2.4	0.0	0.3	0.5	0.0	1.0	2.0	2.5
A Syntax bug	0.6	3.0	0.0	0.3	1.0	0.0	0.2	4.7	2.7
B.1 API misuse	0.0	2.4	0.6	2.4	1.0	1.8	0.5	1.3	1.0
B.2 Definition missing	0.0	5.5	0.0	0.0	0.0	0.0	0.5	2.2	1.8
B.3 Incorrect boundary condition check	0.0	3.0	0.0	1.5	0.5	1.5	2.8	4.3	1.7
B.4 Incorrect argument	0.0	0.0	0.0	1.0	0.0	0.3	1.5	0.3	1.0
B.5 Minors	0.6	0.6	0.6	0.5	0.0	2.3	1.8	1.7	2.2
B Runtime bug	0.6	11.6	1.2	4.0	4.0	2.3	7.2	9.8	7.7
C.1 Misunderstanding and logic error	12.8	20.7	17.1	12.0	15.5	13.8	31.3	46.5	37.5
C.2 Hallucination	0.6	0.0	1.2	1.8	5.8	3.0	7.0	3.0	4.5
C.3 Input/output format error	0.0	0.0	1.2	2.8	0.8	2.8	0.3	2.7	1.7
C.4 Minors	0.0	0.0	0.0	0.0	0.8	0.3	0.7	2.7	2.0
C Functional bug	13.4	20.7	19.5	16.5	22.8	19.8	39.3	54.8	45.7
D Ambiguous problem description	0.0	0.0	0.0	0.0	0.0	0.0	0.8	0.8	0.8

tendenciu generovať kód s bežnými bezpečnostnými zraniteľnosťami, ako je nedostatočná validácia vstupov alebo nebezpečná práca s externými zdrojmi. [25].

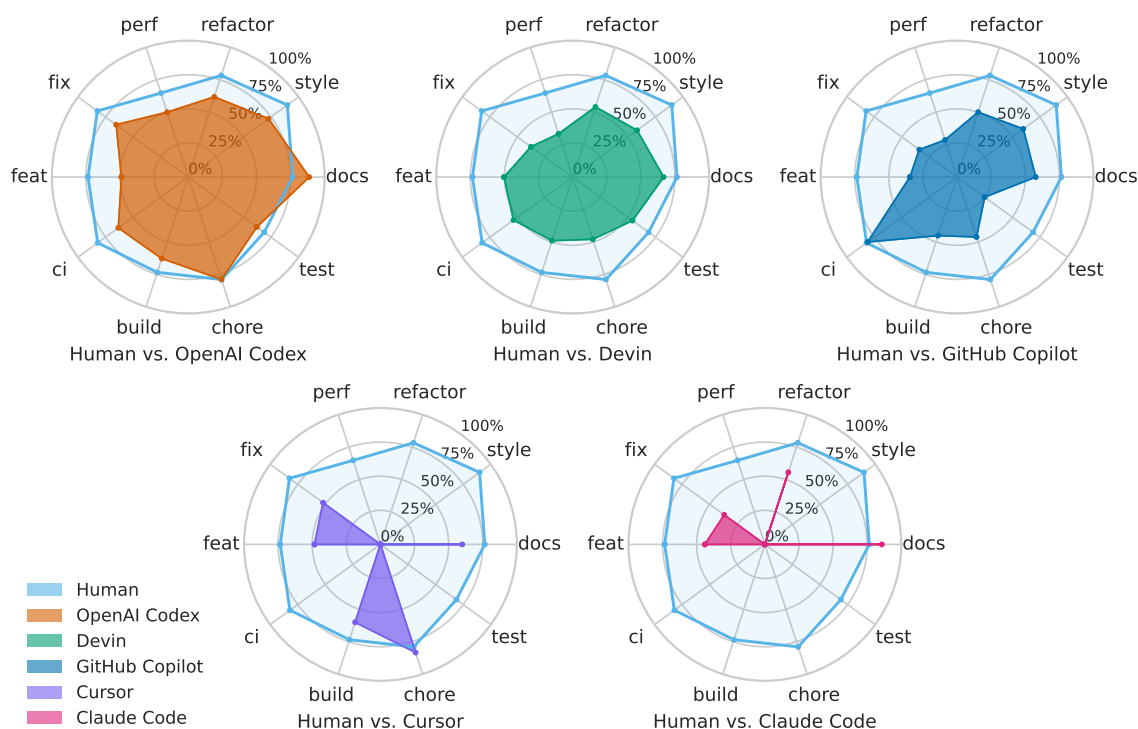
1.1.3 Prístupy k zlepšeniu generovania kódu

S cieľom zmierniť identifikované limity sa výskum zameriava na rôzne techniky zlepšovania generovania kódu. Jednou z najdostupnejších stratégií je promptné inžinierstvo, pri ktorom sa formulácia vstupu využíva na explicitné usmernenie správania modelu [10]. Výskumy ukazujú, že bezpečnostne orientované alebo rolovo definované prompty môžu výrazne znížiť výskyt zraniteľností v generovanom kóde, hoci výsledky nie sú vždy konzistentné naprieč rôznymi úlohami [25].

Ďalším smerom je kontextové učenie (in-context learning), pri ktorom sú modelu poskytnuté ukážkové riešenia priamo v kontexte vstupu. Efektivita tejto techniky však silne závisí od výberu príkladov. Modelovo orientované prístupy, ktoré berú do úvahy správanie konkrétného modelu pri výbere kontextu, dosahujú stabilnejšie a presnejšie výsledky než náhodné alebo heuristické stratégie [16].

Významné zlepšenia prináša aj explicitné štruktúrovanie uvažovania modelu prostredníctvom chain-of-thought promptovaniu. Rozdelenie riešenia na fázy porozumenia problému, návrhu algoritmu a samotnej implementácie vedie k konzistentnejšiemu kódu a znižuje výskyt logických chýb [15].

Analyzované práce sa zhodujú v tom, že žiadna z uvedených techník nepredstavuje univerzálne riešenie. Kvalita generovaného kódu je výsledkom kombinácie vlastností modelu, spôsobu jeho použitia a miery adaptácie na konkrétny kontext [10, 8, 4, 26].



Obr. 1.1: Miera akceptácie žiadostí o prijatie zmien (prevzaté z [14])

1.2 Nástroje pre AI-asistovaný vývoj softvéru

Rýchly rozvoj LLM viedol k vzniku celej triedy nástrojov určených na podporu vývojárov pri tvorbe softvéru. Táto sekcia sa zameriava na praktické nástroje, nie na všeobecné schopnosti modelov. Analýza týchto nástrojov slúži ako východisko pre identifikáciu ich spoločných obmedzení, ktoré motivujú návrh vlastného agentického prístupu v ďalšej časti práce.

1.2.1 Najpoužívanější nástroje pre AI-asistovaný vývoj

Súčasný nástroje pre UI-asistovaný vývoj softvéru možno rozdeliť podľa spôsobu interakcie s vývojárom a miery autonómie. Najrozšírenejšie sú konverzačné systémy a nástroje integrované priamo do vývojových prostredí, pričom v poslednom období sa objavujú aj agentické nástroje schopné vykonávať komplexnejšie pracovné postupy.

ChatGPT a Gemini patria medzi široko používané konverzačné nástroje, ktoré umožňujú generovanie zdrojového kódu na základe prirodzeného jazykového zadania. Ich hlavnou výhodou je flexibilita a schopnosť pracovať s neformálne definovanými požiadavkami. Vývojár môže prostredníctvom dialógu postupne spresňovať zadanie, žiadať alternatívne riešenia alebo vysvetlenie existujúceho kódu. Nevýhodou je absencia priamej integrácie do vývojového prostredia a obmedzený prístup ku globálnemu kontextu rozsiahlejších projektov.

GitHub Copilot predstavuje nástroj pre UI-asistovaný vývoj založený na LLM tré-
novaných na rozsiahlych korpusoch zdrojového kódu a textových dát. Často sa využíva
aj ako nástroj integrovaný do vývojových prostredí, ktorý poskytuje priebežné návrhy
kódu počas písania. V tomto kontexte je optimalizovaný najmä na dopĺňanie existujúcej
implementácie a zrýchlenie rutinných činností.

Osobitnú kategóriu tvoria agentické systémy. Medzi najpoužívanéjšie patria Gemini
CLI a Roo Code. Gemini CLI je open-source agentický nástroj, ktorý je ovládaný pro-
stredníctvom príkazového riadku a dokáže vykonávať shell príkazy, pracovať so súbormi
a interagovať s repozitármi. Roo Code reprezentuje agentický prístup, pri ktorom sys-
tém dokáže iteratívne pracovať s projektom, vykonávať viacero krokov a upravovať kód
bez neustáleho zásahu vývojára, pričom umožňuje nastavenie rôznych rolí podľa typu
použitia. Je navrhnutý ako rozšírenie pre vývojové prostredie Visual Studio Code.

1.2.2 Analýza kvality kódu generovaného AI nástrojmi

ChatGPT a GitHub Copilot patria medzi najvýznamnejšie nástroje v oblasti UI-
asistovaného generovania zdrojového kódu, čo sa odráža aj v ich častom výskyte v
empirických štúdiách. Analýza kvality nimi generovaného kódu sa zameriava na deter-
ministickosť, funkčnú správnosť a komplexnosť riešení.

Deterministickosť a stabilita výstupov Pri opakovanom generovaní riešení pre
rovnaké zadanie vykazuje ChatGPT výraznú variabilitu výstupov. Rozdiely sa preja-
vujú nielen v syntaktickej podobe kódu, ale aj v zvolenom algoritmickom postupe a
výslednom správaní programu [20]. Táto vlastnosť vedie k nízkej reprodukovateľnosti
výsledkov a komplikuje systematické hodnotenie kvality generovaného kódu.

V prípade GitHub Copilot sa pozoruje konzistentnejší výkon pri riešení úloh, pričom
generované riešenia vykazujú nižšiu mieru variability v porovnaní s konverzačnými
modelmi [17]. Napriek tomu sa aj pri tomto nástroji vyskytujú rozdiely v generovaných
riešeníach v závislosti od konkrétneho zadania a jeho formulácie.

Funkčná správnosť Kód generovaný oboma nástrojmi je vo väčšine prípadov syn-
takticky korektný, avšak jeho funkčná správnosť je obmedzená. Pri podrobnejšom tes-
tovaní sa objavujú logické chyby, neúplná implementácia požiadaviek alebo nesprávne
ošetrenie okrajových prípadov [18, 19, 17].

V prípade ChatGPT sú chyby často dôsledkom nesprávneho porozumenia zadania
alebo generovania nepresných algoritmických postupov [20, 18]. GitHub Copilot dosa-
huje porovnateľnú úroveň funkčnej správnosti, pričom jeho výkon závisí od konkrétnej
úlohy a typu problému [17].

Tabuľka 1.3: Porovnanie vlastností kódu generovaného nástrojmi ChatGPT a GitHub Copilot na základe analyzovaných štúdií.

Kritérium	ChatGPT	GitHub Copilot
Deterministickosť	Vysoká variabilita výstupov pri rovnakom zadaní	Nižšia variabilita, relatívne konzistentnejšie návrhy
Funkčná správnosť	Syntakticky korektný kód, avšak časté logické chyby alebo neúplné pokrytie požiadaviek	Variabilná správnosť v závislosti od úlohy a kontextu
Typ chýb	Chyby spôsobené nesprávnym porozumením zadania alebo nevhodným algoritmickým postupom	Chyby závislé od kontextu, často súvisiace s neúplným pokrytím požiadaviek
Komplexnosť kódu	Tendencia generovať rozsiahlejšie a menej optimalizované riešenia	Možný výskyt neefektívnych alebo redundantných implementácií

Komplexnosť a udržiavateľnosť Generovaný kód má tendenciu vykazovať zvýšenú komplexnosť. V riešeniach sa objavujú nadbytočné konštrukcie, redundantné operácie alebo neefektívne algoritmické postupy [18, 19, 17]. Tento jav sa môže zosilniť pri iteratívnom generovaní alebo úpravách kódu, čo vedie k zníženiu čitateľnosti a udržiavateľnosti výsledného riešenia.

ChatGPT často generuje samostatné riešenia, ktoré môžu byť rozsiahlejšie alebo menej optimalizované [18, 19]. GitHub Copilot generuje riešenia, ktoré sú často priamo použiteľné, avšak aj v tomto prípade sa môžu objaviť neefektívne alebo nadbytočné implementácie [17].

Hlavné rozdiely medzi nástrojmi ChatGPT a GitHub Copilot zhrňuje tabuľka 1.3. Generovaný kód je vhodný ako východisko pre ďalší vývoj, avšak jeho použitie si vyžaduje dôkladné testovanie a manuálnu revíziu [20, 18, 19, 6, 17].

1.3 GraphRAG a štruktúrované znalosti

Jedným z kľúčových smerov výskumu zameraného na prekonanie limitov LLM je *Graph Retrieval-Augmented Generation* (GraphRAG). Tento prístup rozširuje klasické schémy retrieval-augmented generation (RAG) o využitie grafovo štruktúrovaných znalostí, ktoré explicitne modelujú entity a vzťahy medzi nimi. [21]. Cieľom tohto prístupu je zvýšiť presnosť, konzistentnosť a kontrolovateľnosť generovaných výstupov prostredníctvom explicitnej práce so znalosťami.

1.3.1 Základný princíp

Architektúra GraphRAG systémov sa typicky skladá z troch hlavných fáz: grafového indexovania, grafovo riadeného vyhľadávania a generovania odpovede obohatenej o grafovú znalosť [21]. V prvej fáze sú vstupné znalosti transformované do grafovej reprezentácie, kde uzly reprezentujú entity a hrany ich vzájomné vzťahy. V druhej fáze je nad týmto grafom vykonané vyhľadávanie relevantných uzlov, ciest alebo podgrafov vzhľadom na vstupný dotaz. Výsledkom je štruktúrovaný kontext, ktorý je následne využitý v generatívnej fáze ako doplnkový vstup pre jazykový model.

Dôležitým aspektom GraphRAG je skutočnosť, že vyhľadávanie nie je obmedzené na jednorazový výber relevantných informácií. Viaceré prístupy umožňujú viackrokové alebo iteratívne prechádzanie grafu, čím podporujú komplexné uvažovanie nad vzťahmi medzi entitami [27, 12].

1.3.2 Prínosy GraphRAG

Hlavnou výhodou GraphRAG je jeho schopnosť explicitne modelovať a využívať relačné znalosti. Kým klasický RAG je založený predovšetkým na podobnosti textových reprezentácií, GraphRAG umožňuje zachytiť viacstupňové závislosti a logické väzby, ktoré sú pre mnohé úlohy kľúčové [21]. To je obzvlášť dôležité v doménach, kde odpoveď závisí od reťazca vzťahov medzi viacerými entitami, a nie od jednej izolovanej informácie. Rozdiel medzi klasickým prístupom RAG a GraphRAG je schematicky znázornený na obrázku 1.2, kde je viditeľný posun od práce s textovými úryvkami k vyhľadávaniu relevantných podgrafov v znalostnej báze.

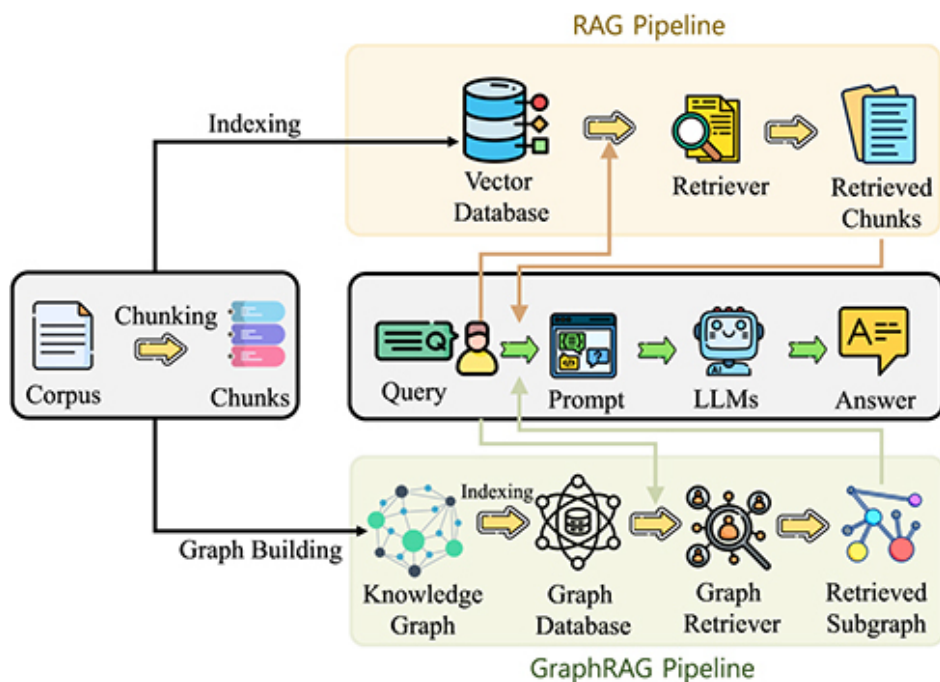
V kontexte generovania zdrojového kódu predstavuje GraphRAG prirodzenú odpoveď na viaceré identifikované limity LLM. Ako poukazujú existujúce štúdie, LLM majú obmedzený globálny kontext a nedisponujú explicitným modelom štruktúry softvérových systémov [10]. Grafové reprezentácie umožňujú modelovať závislosti medzi funkciami, triedami, modulmi alebo aplikačnými rozhraniami a poskytujú tak jazykovému modelu konzistentný a overiteľný kontext.

Ďalším prínosom je flexibilita aktualizácie znalostí. Keďže grafová znalostná báza je oddelená od samotného modelu, je možné ju aktualizovať bez potreby opätovného tréningu alebo jemného doladenia LLM. [27].

1.3.3 Ontológia a reprezentácia znalostí

Ontológia predstavuje explicitnú špecifikáciu konceptualizácie určitej domény. Ide o formálny popis pojmov, ktoré v danej doméne existujú, spolu so vzťahmi medzi nimi.

Poskytujú spoločný slovník a jednotnú štruktúru, ktorá umožňuje konzistentnú interpretáciu informácií naprieč rôznymi aplikáciami. Vďaka formálnej reprezentácii poj-



Obr. 1.2: Porovnanie architektúr klasického RAG a GraphRAG. (prevzaté z [21])

mov a vzťahov umožňujú ontológie efektívnejšie spracovanie znalostí, ich integráciu z rôznych zdrojov a podporu automatizovaného uvažovania nad nimi [7].

V kontexte znalostných báz a GraphRAG systémov môžu ontológie slúžiť ako mechanizmus na organizáciu a prepojenie znalostí do štruktúrovanej formy, ktorá umožňuje efektívnejšie vyhľadávanie relevantných informácií a zachovanie významových vzťahov medzi jednotlivými entitami.

1.3.4 Obmedzenia, riziká a praktické požiadavky

Napriek svojim výhodám prináša GraphRAG aj viaceré výzvy. Jedným z hlavných problémov je škálovateľnosť, keďže efektívne vyhľadávanie v rozsiahlych grafoch si vyžaduje sofistikované indexačné a vyhľadávacie mechanizmy. Ďalším rizikom je závislosť kvality generovania od kvality samotnej grafovej znalostnej bázy. Neúplné alebo nekonzistentné grafy môžu viesť k chybným alebo zavádzajúcim výstupom, a to aj v prípade silného generatívneho modelu [21, 27].

Použitie GraphRAG si zároveň vyžaduje dodatočné systémové komponenty, ako sú nástroje na konštrukciu a údržbu grafu, mechanizmy na mapovanie dotazov do grafového priestoru a stratégie integrácie grafových informácií do generatívneho procesu [21].

1.4 Prípady použitia

Prípady použitia predstavujú spôsob zachytenia funkčných požiadaviek systému prostredníctvom opisu interakcie medzi systémom a jeho aktérmi smerujúcej k dosiahnutiu konkrétneho cieľa [3].

V oblasti softvérového inžinierstva (software engineering, SE) sa ustálili dva významné prístupy k práci s prípadmi použitia, reprezentované Jacobsonom a Cockburnom.

Jacobsonov prístup chápe prípady použitia ako základný prvok vývoja riadeného prípadmi použitia, kde slúžia ako hlavný prostriedok pre analýzu, návrh aj testovanie systému [11]. Prípady použitia sú prepojené s ďalšími modelmi systému a tvoria základ pre jeho architektúru. Dôraz je kladený na ich využitie v celom životnom cykle vývoja a na prepojenie medzi požiadavkami a implementáciou.

Cockburnov prístup sa zameriava na štruktúrovaný textový opis prípadov použitia. Každý prípad použitia obsahuje definovaného aktéra, cieľ, hlavný scenár úspešného priebehu a rozšírenia, ktoré reprezentujú alternatívne alebo chybové situácie [3]. Tento prístup kladie dôraz na zrozumiteľnosť, jednoduchosť a praktické využitie pri komunikácii medzi zainteresovanými stranami.

Hlavný rozdiel medzi týmito prístupmi spočíva v ich zameraní. Jacobsonov prístup využíva prípady použitia ako centrálny prvok riadenia vývoja systému, zatiaľ čo Cockburnov prístup sa sústreďuje na ich formu a kvalitu zápisu. Kým Jacobson zdôrazňuje ich úlohu v celkovom procese vývoja, Cockburn sa orientuje na ich použiteľnosť ako nástroja na zachytenie a komunikáciu požiadaviek.

1.5 Vektorová reprezentácia

Embeddingy predstavujú spôsob reprezentácie slov alebo textových jednotiek vo forme numerických vektorov v spojitom vektorovom priestore [1]. Cieľom tejto reprezentácie je zachytiť význam slov tak, aby slová s podobným významom mali podobné vektorové reprezentácie.

Na rozdiel od tradičných prístupov, ktoré reprezentujú slová diskkrétne (napr. one-hot encoding), embeddingy umožňujú zachytiť sémantické aj syntaktické vzťahy medzi slovami. Vektorové reprezentácie sú získavané na základe distribučnej hypotézy, podľa ktorej majú slová v podobných kontextoch podobný význam [1].

Embeddingy sa učia z veľkých textových korpusov a umožňujú efektívne porovnávanie slov alebo textov pomocou matematických operácií vo vektorovom priestore. Táto vlastnosť je základom pre úlohy ako vyhľadávanie podobnosti, klasifikácia alebo spracovanie prirodzeného jazyka [1].

1.6 Agentické systémy

S postupným posunom LLM od statického generovania výstupov k explicitnému uvažovaniu sa v SE objavuje nová trieda systémov označovaných ako agentické systémy.

1.6.1 Od jazykových modelov k autonómnym agentom

Základným predpokladom vzniku agentických systémov je rozšírenie LLM o mechanizmy uvažovania, pamäte a spätnej väzby [5]. Samotné generovanie odpovedí, typické pre skoršie použitie LLM, sa ukazuje ako nedostatočné pre riešenie komplexných a dlhodobých úloh [5, 22]. Výskum preto smeruje k modelom, ktoré dokážu rozkladať problém na viacero krokov, iteratívne vyhodnocovať vlastné riešenia a adaptovať ďalšie správanie na základe výsledkov predchádzajúcich akcií [5].

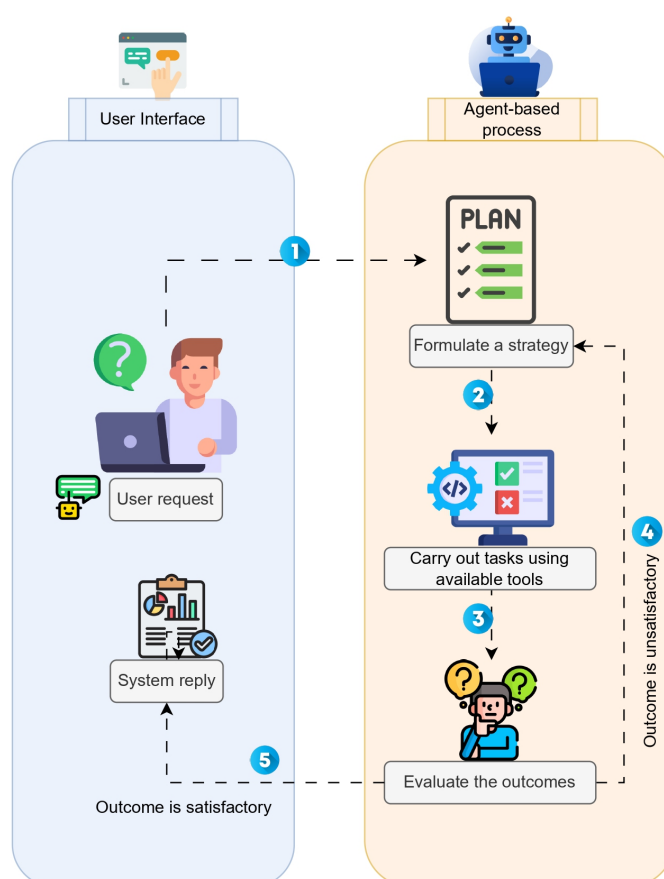
V tomto kontexte sa autonómny agent definuje ako systém, ktorý disponuje cieľom, vnútorným stavom a schopnosťou vykonávať akcie v externom prostredí. Jeho správanie má cyklický charakter, zahŕňajúci vnímanie vstupu, uvažovanie, vykonanie akcie, pozorovanie výsledku a aktualizáciu pamäte. [5]. Agentický cyklus autonómnych systémov, pozostávajúci z plánovania, vykonávania akcií, hodnotenia výsledkov a iteratívnej spätnej väzby, je schematicky znázornený na obr. 1.3.

Zároveň sa ukazuje, že efektívnosť agentických systémov nie je daná výlučne veľkosťou použitého jazykového modelu. Naopak, pre autonómne správanie sú často vhodnejšie menšie, špecializované jazykové modely, ktoré poskytujú vyššiu deterministickosť, nižšiu latenciu a lepšiu kontrolovateľnosť [2].

1.6.2 Multi-agentové systémy v softvérovom inžinierstve

Osobitnú kategóriu agentických systémov tvoria multi-agentové architektúry, v ktorých viacero agentov spolupracuje na riešení spoločného problému. Každý agent môže mať špecializovanú rolu, napríklad plánovanie úloh, generovanie zdrojového kódu, testovanie alebo validáciu výsledkov. Takýto prístup umožňuje paralelizáciu práce a lepšie rozdelenie zodpovedností, čím sa agentické systémy približujú organizácii ľudských vývojových tímov [9].

Výskum ukazuje, že multi-agentové systémy sú v SE použiteľné naprieč viacerými fázami životného cyklu softvéru, od analýzy požiadaviek cez implementáciu až po testovanie a revíziu kódu. Ich hlavnou výhodou je schopnosť riešiť komplexnejšie úlohy, ktoré presahujú možnosti jedného monolitického modelu [9]. Súčasne však rastie zložitosť koordinácie medzi agentmi a zvyšuje sa riziko šírenia chýb medzi jednotlivými komponentmi systému [9, 5].



Obr. 1.3: Schéma agentického pracovného postupu ilustrujúca cyklus plánovania, vykonávania úloh, hodnotenia výsledkov a iteratívneho zlepšovania riešenia v autonómnych agentických systémoch (prevzaté z [5]).

1.6.3 Výhody a obmedzenia agentických prístupov

Medzi hlavné výhody agentických systémov patrí schopnosť autonómneho plánovania, iteratívneho zlepšovania riešení a paralelného spracovania úloh [9, 5]. Tieto vlastnosti umožňujú efektívnejšie zvládanie komplexných problémov a znižujú potrebu neustáleho zásahu vývojára [9]. Agentické systémy tak podporujú posun roly človeka od priamej implementácie k dohľadu, hodnoteniu a riadeniu cieľov [9, 5].

Na druhej strane však takéto prístupy prinášajú aj významné riziká. Nekontrolovaná autonómia môže viesť k nežiadanej správaní, najmä ak sú ciele systému nepresne špecifikované [5, 22]. Chyby vzniknuté v jednom kroku agentického cyklu sa môžu šíriť do ďalších fáz a znižovať kvalitu celého riešenia [5]. Ďalším problémom je absencia štandardizovaných metrík na hodnotenie agentického správania, čo sťažuje objektívne porovnávanie jednotlivých prístupov [9, 22].

Agentické systémy preto nemožno vnímať ako univerzálne riešenie, ale ako ďalší

krok v postupnej evolúcii využitia UI v softvérovom inžinierstve [22, 2]. Ich potenciál spočíva v rozšírení možností automatizácie a podpory vývojárov, nie v úplnej náhrade ľudského rozhodovania [9, 22].

1.7 Cieľ práce, výskumné otázky a hypotézy

1.7.1 Súvisiace práce

V oblasti využitia UI pri vývoji softvéru vzniklo viacero záverečných prác, ktoré sa zameriavajú na generovanie zdrojového kódu alebo podporu vývojového procesu pomocou LLM.

Práca [13] sa zaoberá generovaním zdrojového kódu s využitím prístupu RAG, pričom kombinuje interné a externé zdroje znalostí. Generovanie prebieha ako lineárny proces na základe textového zadania.

Práca [24] sa zameriava na generovanie testov softvérových systémov pomocou LLM v kombinácii s prístupom RAG. Navrhnutý systém využíva pipeline na spracovanie vstupu a generovanie testovacích prípadov.

Na základe uvedených prác možno konštatovať, že existujúce prístupy sa zameriavajú najmä na priame generovanie kódu alebo na inú súčasť tvorenia kódu a to jeho testovanie. Na rozdiel týchto prístupov nami navrhovaný systém využíva viacfázový proces, v ktorom je zadanie najskôr transformované do formálnej reprezentácie a následne rozdelené na menšie časti, ktoré sú generované samostatne a priebežne validované.

1.7.2 Cieľ práce

Cieľom tejto práce je na základe tejto analýzy navrhnuť agentický prístup ku generovaniu zdrojového kódu, ktorý integruje generatívne schopnosti LLM so systematickým využitím externých znalostí, explicitným riadením pracovného postupu a fragmentáciou riešenia na menšie, čiastkové úlohy. Súčasťou navrhovaného prístupu je aj postupná generácia kódu na úrovni jednotlivých metód, ktorá umožňuje lepšiu kontrolu nad procesom tvorby. Navrhovaný prístup sa zameriava na zlepšenie kvality, spoľahlivosti a kontrolovateľnosti generovaného kódu.

Súčasťou práce je implementácia navrhnutého riešenia a jeho experimentálne overenie na praktických scenároch, pričom cieľom je porovnať jeho vlastnosti s tradičným využitím LLM a iným už existujúcim agentickým systémom a vyhodnotiť jeho prínosy a limity.

1.7.3 Výskumné otázky a hypotézy

Na základe cieľov práce boli formulované nasledujúce výskumné otázky (Research Questions, RQ) a hypotézy (Research Hypothesis, RH).

- **RQ1:** Dokážu agentické systémy zlepšiť funkčnú správnosť a kvalitu generovaného zdrojového kódu v porovnaní s priamym generovaním pomocou veľkého jazykového modelu?
 - **RH1.1:** Agentické systémy dosiahnu vyššiu úspešnosť testovacích scenárov a vyššie F1 skóre než priame promptovanie LLM.
 - **RH1.2:** Agentické systémy dosiahnu lepšie hodnotenia kvality kódu pomocou metrík Pylint a Radon než priame promptovanie LLM.
- **RQ2:** Ovplyvňuje využitie GraphRAG znalostnej bázy funkčnú správnosť generovania zdrojového kódu v agentickom systéme?
 - **RH2:** Využitie GraphRAG znalostnej bázy zvýši úspešnosť testovacích scenárov a F1 skóre agentického systému.
- **RQ3:** Dokáže fragmentácia generovania do viacerých krokov a špecializovaných úloh viesť ku väčšej funkčnej správnosti a kvality generovaného zdrojového kódu v porovnaní s inými agentickými prístupmi?
 - **RH3.1:** Navrhovaný agentický systém založený na fragmentácii úloh dosiahne vyššiu úspešnosť testovacích scenárov a vyššie F1 skóre než ostatné porovnávané agentické prístupy.
 - **RH3.2:** Navrhovaný agentický systém založený na fragmentácii úloh dosiahne lepšie hodnotenia kvality kódu pomocou metrík Pylint a Radon než ostatné porovnávané agentické prístupy.

Kapitola 2

Návrh riešenia

Navrhované riešenie predstavuje agentický systém na generovanie zdrojového kódu v programovacom jazyku Python z prirodzeného jazykového zadania, implementovaný pomocou frameworku LangGraph.

Systém využíva kombináciu viacerých princípov, najmä využitie štruktúrovaných znalostí, iteratívnej validácie generovaných výstupov a rozdelením problému na menšie časti.

2.1 Výber jazykového modelu

Výber vhodného LLM predstavuje kľúčový aspekt návrhu agentického systému, keďže tento model tvorí generatívne jadro väčšiny agentov. Pri návrhu riešenia je potrebné zohľadniť nielen kvalitu generovaných výstupov, ale aj praktické faktory, ako sú výpočtové nároky, latencia a dostupnosť modelu.

Jednou z možností je využitie lokálne spúšťaných modelov prostredníctvom nástrojov, ako je napríklad *Ollama*. Tento prístup umožňuje plnú kontrolu nad spracovaním dát a eliminuje závislosť od externých služieb. Nevýhodou sú však vysoké hardvérové nároky, ktoré sa prejavujú zvýšenou latenciou generovania.

Alternatívou sú cloudové modely poskytované prostredníctvom API rozhraní. Tieto modely spravidla dosahujú nižšiu latenciu, avšak ich použitie je limitované cenovou politikou a obmedzeniami počtu požiadaviek.

Vzhľadom na charakter navrhovaného systému, ktorý využíva fragmentáciu problému a sekvenčné generovanie viacerých častí riešenia, je počet volaní LLM výrazne vyšší. Preto je potrebné zvoliť model, ktorý umožňuje dostatočný počet požiadaviek bez výrazných finančných nákladov alebo obmedzení.

Na základe týchto požiadaviek boli pre implementáciu systému zvolené modely poskytované platformou Mistral, konkrétne model *Mistral Large* pre všeobecné úlohy a model *Codestral* pre generovanie zdrojového kódu. Táto voľba bola motivovaná kombi-

Na základe prípadu použitia systém pokračuje návrhom architektúry, ktorý zahŕňa definovanie tried, metód a atribútov. Tento návrh je opäť podrobený revízii, aby sa overilo pokrytie požiadaviek a správnosť navrhnutej štruktúry.

Dôležitou súčasťou architektúry je znalostná báza realizovaná pomocou prístupu GraphRAG. Táto báza obsahuje informácie o existujúcich riešeniach a je využívaná pri vyhľadávaní relevantných znalostí pre generovanie kódu jednotlivých metód. Znalostná báza je priebežne a automaticky dopĺňaná prostredníctvom externých zdrojov, konkrétne GitHub repozitárov.

Samotné generovanie zdrojového kódu prebieha postupne na úrovni jednotlivých metód. Agent generovania kódu vytvára implementáciu na základe dostupného kontextu, pričom každý vygenerovaný prvok je následne overený validačným agentom na úrovni metód a na úrovni tried. Tento proces zabezpečuje syntaktickú správnosť a správne pokrytie požiadaviek a zodpovedností generovaných častí programu.

Po vygenerovaní jednotlivých častí nasleduje ich integrácia do výsledného programu. Tento krok je realizovaný agentom spájania kódu metód a tried a následne kontrolovaný validačným agentom, ktorý overuje konzistenciu a požadované správanie celého systému.

Výsledný program je doplnený o diagram aktivít, ktorý slúži na lepšiu interpretáciu správania systému používateľovi. Tento diagram je taktiež predmetom revízie.

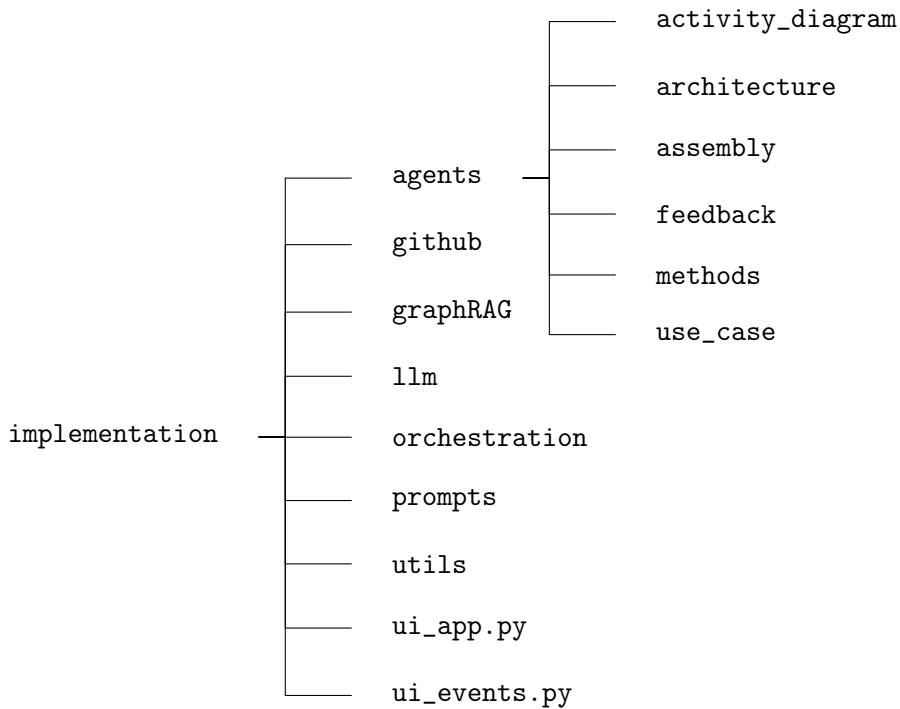
Architektúra systému zahŕňa aj spätnú väzbu od používateľa, ktorá predstavuje dôležitý prvok iteratívneho zlepšovania. Na základe typu spätnej väzby systém rozhoduje, či je potrebné upraviť pôvodný prípad použitia alebo regenerovať konkrétne časti kódu. Tento mechanizmus umožňuje efektívne reagovať na zmeny požiadaviek aj opravu chýb. Ak výstup vyhovuje používateľovi, následne sa môže rozhodnúť, či si praje daný program uložiť do znalostnej bázy.

Navrhovaná architektúra tak predstavuje ucelený proces, ktorý kombinuje formalizáciu zadania, využitie znalostí, postupné generovanie a viacúrovňovú validáciu v rámci jedného integrovaného systému.

2.3 Elektronická príloha implementácie

Súčasťou práce je elektronická príloha vo forme verejného GitHub repozitára, ktorý obsahuje implementáciu navrhovaného agentického systému, GraphRAG znalostnú bázu a podporné komponenty použité pri realizácii práce.

Implementačná časť repozitára je rozdelená do viacerých modulov podľa ich funkcionality. Hlavná štruktúra implementácie je znázornená na obr. 2.2.



Obr. 2.2: Štruktúra implementačnej časti repozitára

Adresár **agents** obsahuje implementáciu jednotlivých agentov zodpovedných za generovanie a validáciu artefaktov systému, ako sú prípady použitia, architektúra, diagramy aktivít alebo výsledný zdrojový kód.

Modul **github** zabezpečuje získavanie a spracovanie implementačných príkladov z GitHub repozitárov použitých pri budovaní znalostnej bázy.

Adresár **graphRAG** obsahuje komponenty zabezpečujúce komunikáciu so znalostnou bázou, vykonávanie GraphRAG dopytov nad grafovou databázou Neo4j a RDF reprezentáciu ontológie použitej na štruktúrovanie znalostí systému.

Komponenty zodpovedné za bezpečné a správne vykonávanie volaní veľkého jazykového modelu sú implementované v adresári **llm**.

Adresár **orchestration** obsahuje komponenty zabezpečujúce koordináciu a prepojenie jednotlivých agentov v rámci pracovného postupu systému.

V adresári **prompts** sa nachádzajú systémové prompty používané pri komunikácii s veľkým jazykovým modelom.

Pomocné funkcionality systému sú implementované v adresári **utils**, zatiaľ čo súbory **ui_app.py** a **ui_events.py** zabezpečujú používateľské rozhranie systému.

2.4 Agentický pracovný postup

V tejto sekcii je podrobnejšie opísaný pracovný postup a implementácia navrhovaného systému z pohľadu jednotlivých fáz spracovania. Každá fáza reprezentuje konkrétnu

časť agentického systému a zodpovedá za spracovanie určitého aspektu generovania zdrojového kódu.

2.4.1 Generovanie prípadu použitia

Prvou fázou navrhovaného systému je transformácia neformálneho vstupného zadania v prirodzenom jazyku do štruktúrovanej reprezentácie vo forme prípadu použitia, ktorá je využívaná v ďalších fázach návrhu.

Voľba reprezentácie požiadaviek V softvérovom inžinierstve existuje viacero spôsobov reprezentácie požiadaviek, napríklad vo forme funkčných a nefunkčných požiadaviek alebo pomocou prípadov použitia. Druhý spomínaný prístup je vhodnejší pre následné generovanie architektúry a implementácie.

V navrhovanom riešení je použitá Cockburnova forma prípadu použitia, ktorá je textová a štruktúrovaná. Oproti Jacobsonovej notácii nie je viazaná na UML diagramy, čo ju robí vhodnejšou pre spracovanie jazykovými modelmi.

Základná štruktúra Cockburnovej notácie bola v rámci tejto práce rozšírená o explicitnú sekciu požiadaviek. Dôvodom je skutočnosť, že nie všetky informácie zo vstupného zadania je možné prirodzene zaradiť do scenárov alebo systémových zodpovedností. Pridaním tejto sekcie sa zabezpečuje, že jazykový model zachytí aj doplnkové požiadavky, ktoré by inak mohli byť stratené.

Výsledný prípad použitia preto pozostáva z nasledujúcich častí:

- **Use Case** – názov prípadu použitia,
- **Primary Actor** – hlavný aktér,
- **Goal** – cieľ prípadu použitia,
- **System Responsibilities** – zodpovednosti systému,
- **Preconditions** – vstupné podmienky,
- **Postconditions** – výstupné podmienky,
- **Main Success Scenario** – hlavný scenár,
- **Extensions** – alternatívne scenáre,
- **Special Requirements** – doplnkové požiadavky a obmedzenia.

Generovanie pomocou LLM Generovanie prípadu použitia je realizované agentom, ktorý využíva LLM. Vstupom je používateľské zadanie, na základe ktorého sa vytvára štruktúrovaný prompt.

Pri návrhu promptu boli použité techniky štruktúrovania, rolovej definície a kontextového učenia, pričom ich konkrétna realizácia a krátke úryvky jednotlivých častí sú znázornené v ukážke 2.1.

```
ROLE: Senior software engineer specialized in requirements
      analysis
TASK: Transform the user task into a structured Use Case ...
INPUT: User Task
PRIORITY: Ensure completeness of system behavior ...
REQUIREMENT EXTRACTION RULES: MUST include all explicitly stated
      requirements ...
SECTION RULES: Goal = main objective of the actor ...
FORBIDDEN: class/method descriptions ...
REFERENCE EXAMPLE:
Use Case: Draw Circle on Canvas
Primary Actor: User
Goal: Create a circle on the drawing canvas.
System Responsibilities: Maintain the state of the drawing
      canvas ...
Preconditions: The application window is open ...
Postconditions: A circle is displayed on the canvas at the
      selected position.
Main Success Scenario:
1. User selects the circle drawing tool ...
Extensions: 2a. Click occurs outside the canvas -> System
      ignores the input.
Special Requirements: The circle should be clearly visible on
      the canvas ...
OUTPUT FORMAT: Preserve exact section structure from the
      reference ...
```

Ukážka 2.1: Štruktúra promptu pre generovanie prípadu použitia

Validácia a iteratívne zlepšovanie Proces generovania je doplnený validačným agentom, ktorý hodnotí kvalitu prípadu použitia. Validácia prebieha opäť pomocou jazykového modelu, pričom výstup má štruktúru obsahujúcu skóre kvality v intervale $\langle 0,1 \rangle$ a zoznam identifikovaných problémov. Ukážka 2.2 zobrazuje časť validačného promptu definujúcu pravidlá hodnotenia kvality.

SCORING RULES:

- 1.00 -> no defects
- 0.90-0.99 -> only minor clarifications compared to the user task
- 0.60-0.89 -> noticeable defects affecting correctness or completeness of the user task
- <0.60 -> major missing or incorrect functionality

SCORING CONSTRAINT:

- If ANY required functionality from the user task is missing or incorrect -> score < 0.90
- If ALL requirements and functionality from the user task are correctly represented -> score >= 0.90
- Score MUST reflect severity AND number of defects

Ukážka 2.2: Pravidlá hodnotenia kvality prípadu použitia

Skóre je následne využité na riadenie iterácií, kde ako hraničná hodnota pre pokračovanie v ďalších iteráciách je použitá hodnota 0,9. Maximálny počet validačných krokov je obmedzený na tri iterácie, čo predstavuje kompromis medzi kvalitou a stabilitou. Pri vyššom počte iterácií sa totiž začali objavovať halucinácie validačného agenta, ktorý identifikoval neexistujúce chyby.

Revízia a spätná väzba Na základe výsledku validácie systém rozhoduje o ďalšom postupe. V prípade nedostatočného skóre je prípad použitia upravený len v rozsahu identifikovaných problémov.

Súčasťou návrhu je aj rozlíšenie režimov generovania a revízie, konkrétne počiatkové generovanie, regenerovanie po spätnej väzbe a regenerovanie po revízii. Analogicky sú definované aj režimy revízie, kde systém rozlišuje medzi revíziou počiatkového návrhu a revíziou na základe spätnej väzby používateľa. Používateľská spätná väzba vedie k úprave prípadu použitia iba v prípade, že mení požiadavky, zatiaľ čo implementačné problémy sú riešené až v neskorších fázach. Pre každý z týchto režimov sú použité samostatné prompty, ktoré reflektujú aktuálny kontext a cieľ danej fázy.

Výsledkom tejto fázy je formálne štruktúrovaná reprezentácia požiadaviek, ktorá minimalizuje nejednoznačnosť vstupu a vytvára stabilný základ pre ďalšie fázy generovania.

2.4.2 Návrh architektúry systému

Po vytvorení validovaného prípadu použitia nasleduje fáza návrhu architektúry systému. Cieľom tejto fázy je transformovať scenárovú reprezentáciu požiadaviek na štruktúrovaný návrh softvérového riešenia.

Reprezentácia architektúry Navrhovaná architektúra je reprezentovaná v štruktúrovanej podobe a svojím charakterom sa približuje UML diagramom, konkrétne diagramu tried, keďže explicitne modeluje triedy, ich atribúty a metódy. Oproti klasickým diagramom však ide o textovú a strojovo spracovateľnú reprezentáciu vo formáte JSON.

Architektúra pozostáva zo zoznamu tried, pričom každá trieda obsahuje:

- **name** – názov triedy,
- **responsibility** – stručný popis zodpovednosti triedy,
- **attributes** – zoznam atribútov reprezentujúcich stav systému,
- **methods** – zoznam metód implementujúcich správanie systému.

A v jednotlivých triedach každá metóda obsahuje popis správania, kroky vykonávania, prípadné špeciálne požiadavky, vstupy a výstupy zahrňujúce názov, typ a opis premennej. Takto definovaná štruktúra zabezpečuje prepojenie medzi požiadavkami a implementáciou a slúži ako základ pre generovanie zdrojového kódu.

Generovanie architektúry pomocou LLM Návrh architektúry je realizovaný prostredníctvom špecializovaného agenta, ktorý využíva LLM. Vstupom pre tento proces je validovaný prípad použitia spolu s pôvodným používateľským zadáním.

Pri návrhu promptu sa kladie dôraz najmä na úplné pokrytie požiadaviek a ich mapovanie na jednotlivé metódy. Taktiež každý krok hlavného scenára musí byť reprezentovaný v návrhu architektúry a musí byť vysledovateľný v definovaných metódach.

V prípade, že model vráti nevalidnú štruktúru, je výsledok odmietnutý a generovanie sa opakuje.

Validácia architektúry Podobne ako pri generovaní prípadu použitia, aj návrh architektúry je doplnený validačným agentom. Tento agent overuje, či architektúra správne pokrýva všetky požiadavky definované v prípade použitia a či jednotlivé metódy reprezentujú požadované správanie systému.

Validácia sa zameriava najmä na:

- úplnosť pokrytia požiadaviek,
- správne mapovanie scenárov na metódy,

- konzistenciu stavu systému reprezentovaného atribútmi.

Výstupom validácie je skóre kvality a zoznam identifikovaných problémov, pričom proces je riadený rovnakým mechanizmom iteratívneho zlepšovania ako v predchádzajúcej fáze.

Revízia a spätná väzba Na základe výsledkov validácie je architektúra upravovaná len v rozsahu identifikovaných problémov. Pri revízii sa uplatňuje princíp minimálnych zmien, pričom existujúca architektúra je považovaná za referenčný základ a upravujú sa len nevyhovujúce časti.

Analogicky ako v predchádzajúcej fáze sú definované rôzne režimy generovania a revízie, pre ktoré sú použité samostatné prompty. Tento prístup umožňuje presné riadenie zmien a zachovanie stability návrhu aj pri iteratívnom spracovaní.

Výsledkom tejto fázy je konzistentný návrh architektúry systému, ktorý explicitne mapuje požiadavky na jednotlivé triedy a metódy a predstavuje priamy vstup pre následné generovanie zdrojového kódu.

2.4.3 Znalostná báza (GraphRAG)

Súčasťou navrhovaného systému je externá znalostná báza, ktorá slúži na ukladanie a vyhľadávanie implementačných znalostí získaných z existujúcich zdrojových kódov. Táto báza je realizovaná ako grafová databáza doplnená o mechanizmy sémantického vyhľadávania.

Ontologický model Znalostná báza je založená na ontológii, ktorá modeluje základné prvky softvérového systému a vzťahy medzi nimi. Ontológia je konceptuálne definovaná pomocou jazyka RDFS, čo umožňuje explicitne špecifikovať triedy a ich vzťahy.

Základné triedy ontológie sú:

- **Class** – reprezentuje triedu v systéme,
- **Method** – reprezentuje metódu,
- **Attribute** – reprezentuje atribút triedy,
- **Data** – reprezentuje vstupné a výstupné dáta.

Medzi hlavné vzťahy patria:

- **belongsTo** – väzba medzi metódou a triedou,

- **hasAttribute** – priradenie atribútov k triede,
- **reads, writes** – prístup metódy k atribútom,
- **consumes, produces** – vstupy a výstupy metódy.

Ukážka 2.3 zobrazuje časť ontológie zapísanú pomocou RDFS.

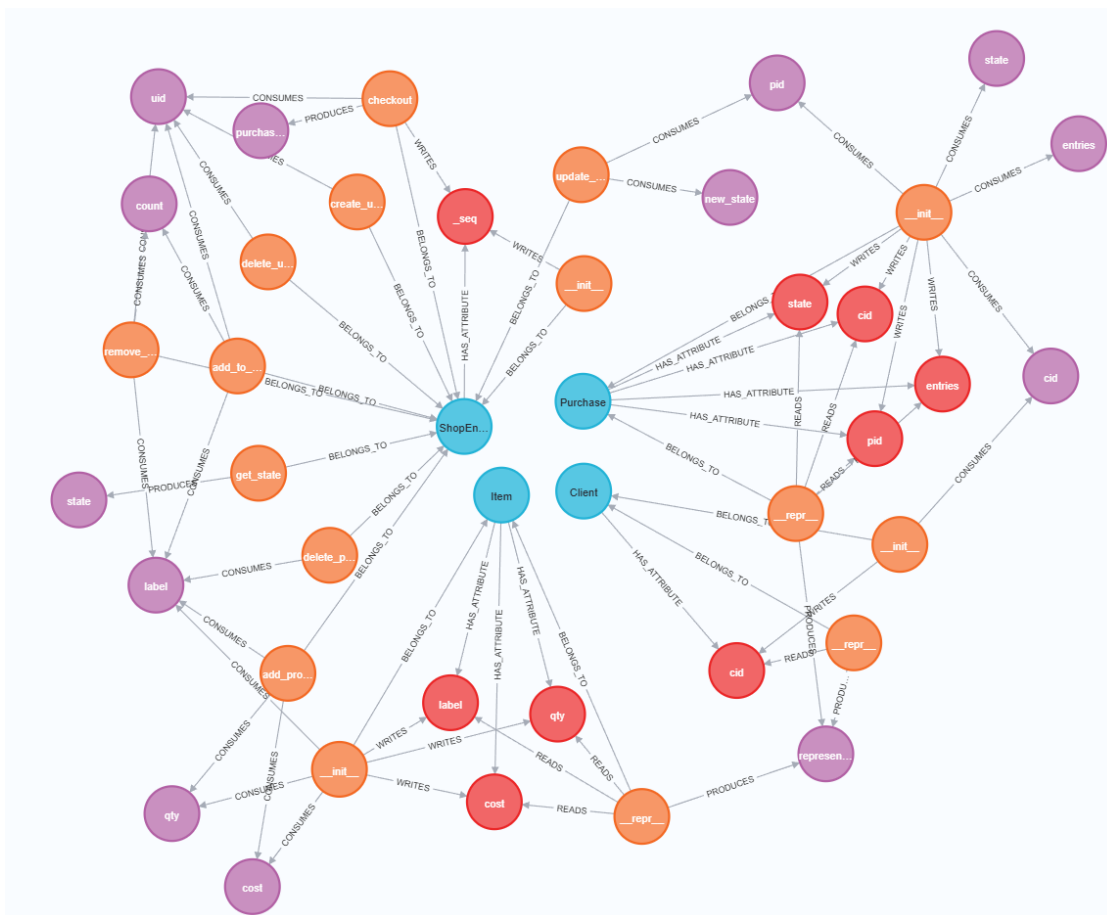
```
:Class rdf:type rdfs:Class .
:Method rdf:type rdfs:Class .
:Attribute rdf:type rdfs:Class .
:Data rdf:type rdfs:Class .

:belongsTo rdf:type rdf:Property ;
    rdfs:domain :Method ;
    rdfs:range :Class .
:hasAttribute rdf:type rdf:Property ;
    rdfs:domain :Class ;
    rdfs:range :Attribute .
:reads rdf:type rdf:Property ;
    rdfs:domain :Method ;
    rdfs:range :Attribute .
:consumes rdf:type rdf:Property ;
    rdfs:domain :Method ;
    rdfs:range :Data .
:name rdf:type rdf:Property ;
    rdfs:domain :Class ;
    rdfs:range rdfs:Literal .
```

Ukážka 2.3: Časť RDFS reprezentácie ontológie

Táto ontológia slúži ako formálny model dát uložených v znalostnej báze. V implementácii je následne mapovaná na grafovú databázu Neo4j, kde sú jednotlivé entity reprezentované ako uzly a vzťahy ako orientované hrany. Ukážka 2.3 zobrazuje konkrétnu podobu znalostnej bázy, kde modré uzly reprezentujú triedy, oranžové uzly metódy, červené uzly atribúty a fialové uzly vstupné a výstupné dáta.

Získavanie znalostí z repozitárov Znalostná база je priebežne dopĺňaná analýzou GitHub repozitárov. Systém periodicky kontroluje stav sledovaných repozitárov na úrovni commitov a identifikuje zmeny oproti predchádzajúcemu spracovanému stavu. V prípade detekcie nového commitu sú spracované iba zmenené súbory, čím sa minimalizuje redundantné spracovanie.



Obr. 2.3: Ukážka grafovej reprezentácie znalostnej bázy v Neo4j.

Spracovanie prebieha na úrovni jednotlivých Python súborov, pričom systém rozlišuje medzi pridanými, upravenými, premenovanými a odstránenými súbormi. V prípade odstránenia alebo premenovania sú príslušné uzly a vzťahy z grafovej databázy odstránené, čím sa zabezpečuje konzistencia znalostnej bázy.

Zdrojový kód je analyzovaný pomocou syntaktickej analýzy konkrétne pomocou štandardného modulu `ast` z jazyka Python, ktorý umožňuje parsovanie zdrojového kódu do abstraktného syntaktického stromu (AST). Na základe AST sa extrahujú jednotlivé funkcie spolu s ich kontextom, vrátane príslušnosti k triede, atribútov triedy a ich využitia. Pre každú funkciu sú identifikované:

- vstupné parametre,
- výstupné hodnoty,
- čítané a zapisované atribúty,
- zdrojový kód funkcie.

Na základe týchto informácií je vytvorená štruktúrovaná reprezentácia funkcie, ktorá je následne obohatená pomocou jazykového modelu. LLM dopĺňa sémantické

metadáta, ako sú stručné zhrnutie funkcie, detailný opis správania a sekvencia krokov vykonávania.

Pred uložením do znalostnej bázy je každá funkcia podrobená filtrovaniu, ktorého cieľom je eliminovať triviálne alebo nerelevantné implementácie. Tento krok zabezpečuje, že databáza obsahuje najmä funkcie s netriviálnou logikou, ktoré majú potenciál byť opakovane využité.

Výsledné informácie sú následne uložené do grafovej databázy Neo4j, kde sú reprezentované ako uzly a vzťahy podľa definovanej ontológie. Tento proces zabezpečuje automatické a priebežné rozširovanie znalostnej bázy na základe reálnych implementácií.

Sémantické vyhľadávanie Vyhľadávanie relevantných znalostí je realizované pomocou embeddingov a kosínusovej podobnosti. Každá metóda uložená v databáze je reprezentovaná vektorom, ktorý vzniká transformáciou textového popisu metódy pomocou embedding modelu z knižnice `sentence-transformers` (`SentenceTransformer`), ktorý poskytuje predtrénované modely na transformáciu textu do vektorového priestoru vhodného pre výpočet kosínusovej podobnosti.

Pre vyhľadávanie podobných metód sa vytvára embedding pre cieľovú metódu a následne sa počíta podobnosť s embeddingami uloženými v databáze.

Kosínusová podobnosť medzi dvoma vektormi **A** a **B** je definovaná ako:

$$\text{sim}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} \quad (2.1)$$

Výsledkom je hodnota v intervale $[-1, 1]$, ktorá vyjadruje mieru podobnosti medzi dvoma reprezentáciami.

Ukážka 2.4 zobrazuje implementáciu výberu podobných metód na základe kosínusovej podobnosti. Funkcia najprv vytvorí embedding cieľovej metódy na základe jej názvu, sumarizácie, textového opisu a krokov funkcionality, následne vypočíta podobnosť so všetkými metódami v databáze a aplikuje prahovanie a zoradenie výsledkov.

```
def select_similar_functions(target_func: dict, graph_functions:
    list, graph_embeddings: list) -> list:
    if len(graph_embeddings) == 0:
        return []

    target_text = build_target_text(target_func)
    target_emb = embedding_model.encode([target_text])[0]

    similarities = cosine_similarity([target_emb],
        graph_embeddings)[0]
    scored = list(zip(graph_functions, similarities))
```

```
THRESHOLD = 0.7
filtered = [(f, score) for f, score in scored if score >=
            THRESHOLD]

if not filtered:
    return []

filtered.sort(key=lambda x: x[1], reverse=True)

MAX_RESULTS = 2
top = filtered[:MAX_RESULTS]

return [{"class": f["class_name"], "name": f["method_name"]}
        for f, score in top]
```

Ukážka 2.4: Výber podobných metód na základe kosínusovej podobnosti

Implementácia využíva pevne definovaný prah podobnosti a obmedzuje počet výsledkov na najrelevantnejšie metódy. Tento prístup zabezpečuje, že do ďalších fáz generovania vstupujú iba najvhodnejšie príklady, čím sa znižuje šum v kontexte a zvyšuje kvalita generovaného kódu.

Výsledkom tejto fázy je množina relevantných implementačných príkladov, ktoré sú využité ako kontext pri generovaní jednotlivých metód a prispievajú k zvýšeniu kvality a konzistencie generovaného riešenia.

2.4.4 Generovanie metód

Táto fáza predstavuje jadro navrhovaného systému, v ktorom dochádza k samotnému generovaniu zdrojového kódu. Generovanie neprebíha jednorazovo pre celý program, ale postupne na úrovni jednotlivých metód, čím sa znižuje komplexita problému, zvyšuje kontrola nad výsledkom a využíva sa silná stránka LLM, čo je generovanie stredne rozsiahlych alebo jednoduchých funkcií. Týmto prístupom implementujeme princíp fragmentácie.

Vstup do fázy Vstupom do generovania metód je návrh architektúry obsahujúci triedy, atribúty a špecifikácie jednotlivých metód. Každá metóda je definovaná pomocou popisu správania, krokov vykonávania, vstupov a výstupov.

Pre každú metódu je pred generovaním zostavený kontext, ktorý obsahuje:

- špecifikáciu metódy,

- atribúty triedy,
- ostatné metódy v triede,
- relevantné príklady zo znalostnej bázy (GraphRAG).

Tento kontext je následne využitý pri generovaní pomocou LLM.

Generovanie pomocou LLM Generovanie jednej metódy je realizované pomocou špecializovaného promptu, ktorý definuje presné pravidlá implementácie. Prompt kladie dôraz na úplné pokrytie špecifikácie, izoláciu zodpovednosti a konzistentné využitie atribútov a metód triedy.

Samotné generovanie prebieha iteratívne pre všetky metódy definované v architektúre, pričom každá metóda je generovaná nezávisle.

Režimy generovania Systém rozlišuje viacero režimov generovania metód v závislosti od aktuálneho stavu:

- **Initial generation** – počiatočné generovanie všetkých metód,
- **Regeneration after validation** – regenerovanie iba nevalidných metód,
- **Feedback revision** – regenerovanie metód ovplyvnených spätnou väzbou.

Výber režimu je riadený stavom systému a počtom validačných iterácií. Ako v predchádzajúcich častiach, validujeme maximálne trikrát. Tento prístup umožňuje minimalizovať počet volaní LLM a zároveň zachovať konzistenciu generovaného kódu.

Validácia metód Každá vygenerovaná metóda je podrobená viacstupňovej validácii. Prvým krokom je syntaktická kontrola pomocou AST analýzy, ktorá overuje, či je kód validný Python program.

Táto kontrola predstavuje rýchly a deterministický spôsob overenia základnej správnosti kódu ešte pred jeho ďalším spracovaním.

Následne prebieha validácia pomocou LLM, ktorá kontroluje súlad implementácie so špecifikáciou. Validácia zahŕňa:

- pokrytie všetkých požadovaných krokov,
- správne použitie vstupov a výstupov,
- konzistentné využitie atribútov a metód,
- logickú správnosť implementácie.

Okrem validácie jednotlivých metód je implementovaná aj validačná fáza na úrovni tried pomocou LLM, ktorá kontroluje konzistenciu medzi metódami a atribútmi.

Iteratívne zlepšovanie V prípade zistenia chýb systém nevalidné metódy automaticky regeneruje. Regenerovanie je riadené validačnými výstupmi a je obmedzené maximálnym počtom troch iterácií.

Pri regenerovaní je využívaný špeciálny prompt, ktorý obsahuje pôvodnú implementáciu a identifikované chyby. Tento prístup umožňuje aplikovať iba minimálne potrebné zmeny, čím sa zachováva stabilita kódu.

Revízia na základe spätnej väzby Systém umožňuje regenerovanie metód aj na základe spätnej väzby používateľa. Tento proces však nie je realizovaný plošne, ale selektívne, s cieľom minimalizovať zásahy do existujúcej implementácie.

Výber metód na regenerovanie prebieha kombináciou deterministickej analýzy a využitia jazykového modelu. Najskôr sú identifikované:

- **nové metódy**, ktoré sa v architektúre objavili a ešte nemajú implementáciu,
- **metódy so zmenenou špecifikáciou**, kde došlo k zmene popisu, vstupov, výstupov alebo krokov vykonávania,
- **odstránené metódy**, ktoré sú následne vyradené z výsledného programu.

Okrem toho je aplikovaná aj analýza dopadu spätnej väzby pomocou LLM, ktorý identifikuje metódy, ktorých správanie je potrebné upraviť na základe používateľských požiadaviek.

Výsledná množina metód na regenerovanie je teda zjednotením týchto troch prístupov: detekcie zmien v architektúre, analýzy špecifikácie a analýzy spätnej väzby. Regenerované sú iba tie metódy, ktorých implementácia musí byť upravená.

Tento prístup zabezpečuje efektívne riadenie zmien, minimalizuje počet volaní LLM a zároveň zabraňuje zbytočnému prepisovaniu správnych častí implementácie.

Integrácia so znalostnou bázou Pri generovaní metód je využívaný kontext zo znalostnej bázy (GraphRAG), ktorý obsahuje podobné implementácie získané zo zdrojových repozitárov. Tento kontext slúži ako inšpirácia pre generovanie kódu, pričom nie je striktno záväzný kvôli jedinečnosti každého zadania a výnimočnosti každej situácie, kedy sa daná metóda implementuje.

Použitie externých znalostí umožňuje generovať konzistentnejšie a kvalitnejšie implementácie, najmä pri komplexnejších metódach.

Výsledkom tejto fázy je množina implementovaných a validovaných metód, ktoré tvoria základ pre zostavenie výsledného programu v nasledujúcej fáze.

2.4.5 Integrácia programu a validácia

Po vygenerovaní jednotlivých metód nasleduje fáza ich integrácie do výsledného programu. Cieľom tejto fázy je zostaviť konzistentný a spustiteľný celok, ktorý zodpovedá definovanej architektúre a zároveň implementuje požadované správanie zo špecifikácie.

Zostavenie programu Integrácia programu je realizovaná špecializovaným agentom, ktorý využíva LLM na zostavenie finálneho kódu. Vstupom do tejto fázy je:

- návrh architektúry,
- implementácie jednotlivých metód,
- prípad použitia.

Agent je zodpovedný za:

- správne priradenie metód do tried,
- inicializáciu atribútov,
- zabezpečenie konzistentného dátového toku,
- vytvorenie riadiaceho toku programu (napr. `main()`).

Pri integrácii je kladený dôraz na zachovanie architektúry a minimalizáciu zásahov do vygenerovaných metód.

Režimy integrácie Podobne ako v predchádzajúcich fázach, aj integrácia programu prebieha v rôznych režimoch:

- **Initial generation** – prvotné zostavenie programu,
- **Regeneration after validation** – oprava programu na základe validačných chýb,
- **Feedback revision** – úprava programu na základe spätnej väzby používateľa.

Každý režim využíva samostatný prompt, ktorý reflektuje aktuálny kontext a cieľ úpravy programu.

Syntaktická a sémantická validácia Po zostavení programu nasleduje jeho validácia. Prvým krokom je syntaktická kontrola, ktorá využíva analýzu pomocou AST a kompilácie programu.

Následne prebieha sémantická validácia, ktorá kombinuje dva prístupy:

- **LLM spätná väzba** – hodnotenie súladu programu s prípadom použitia,
- **embedding porovnanie** – výpočet podobnosti medzi pôvodným a odvodeným prípadom použitia.

Pri druhom prístupe je z vygenerovaného programu najskôr odvodený prípad použitia pomocou LLM, ktorý je následne porovnaný s pôvodným prípadom použitia pomocou embeddingov a kosínusovej podobnosti. V prípade podobnosti menšej ako 0,75 sa program považuje za sémanticky nevalidný.

Ukážka 2.5 zobrazuje implementáciu výpočtu podobnosti medzi dvoma prípadmi použitia, kde sa jednotlivé časti (cieľ, zodpovednosti a hlavný scenár) porovnávajú samostatne a následne sa agregujú pomocou váženého súčtu.

```
def compute_use_case_similarity(uc1: dict, uc2: dict) -> float:
    weights = {
        "goal": 0.3,
        "responsibilities": 0.3,
        "main_flow": 0.4 }
    score = 0

    if uc1["goal"] and uc2["goal"]:
        score += weights["goal"] * cosine_sim(
            embed(uc1["goal"]),
            embed(uc2["goal"]) )

    if uc1["responsibilities"] and uc2["responsibilities"]:
        score += weights["responsibilities"] * cosine_sim(
            embed(" ".join(uc1["responsibilities"])),
            embed(" ".join(uc2["responsibilities"]))) )

    if uc1["main_flow"] and uc2["main_flow"]:
        score += weights["main_flow"] * cosine_sim(
            embed(" ".join(uc1["main_flow"])),
            embed(" ".join(uc2["main_flow"]))) )

    return score
```

Ukážka 2.5: Výpočet podobnosti medzi prípadmi použitia

Tento prístup umožňuje zohľadniť rôzne aspekty správania systému a poskytuje kvantitatívnu mieru zhody medzi požadovanou a implementovanou funkcionalitou.

Testovanie Ak program prejde základnou validáciou, nasleduje fáza testovania. Tá zahŕňa:

- generovanie jednotkových testov pre jednotlivé metódy,
- generovanie integračných testov pre celý systém,
- spúšťanie testov v izolovanom prostredí.

Nie všetky vygenerované metódy sú vhodné na jednotkové testovanie. Z tohto dôvodu systém obsahuje mechanizmus na identifikáciu tzv. testovateľných metód, čím sa predchádza generovaniu nerelevantných alebo nevykonateľných testov.

Rozhodovanie o testovateľnosti je realizované kombináciou heuristických pravidiel a jazykového modelu.

V prvej fáze sa aplikujú jednoduché heuristiky založené na analýze zdrojového kódu. Metódy, ktoré obsahujú operácie závislé od externého prostredia, ako napríklad práca s grafickým rozhraním, udalosťami alebo vykresľovaním, sú automaticky vyradené. Tento krok slúži na rýchle odfiltrovanie zjavne netestovateľných metód.

Následne je na zostávajúce metódy aplikované rozhodovanie pomocou LLM, ktorý posudzuje testovateľnosť na základe špecifikácie a implementácie metódy.

Testy sú následne spúšťané v samostatnom procese, čím sa zabezpečuje izolácia a bezpečnosť vykonávania.

Použitie samostatného procesu zabraňuje ovplyvneniu hlavného systému chybami v testovanom kóde a umožňuje bezpečné vykonávanie aj potenciálne nebezpečných operácií.

Iteratívne zlepšovanie V prípade zistenia chýb počas validácie alebo testovania systém automaticky prechádza do režimu regenerácie programu. Regenerovanie je riadené identifikovanými chybami a prebieha s dôrazom na minimálne potrebné úpravy.

Proces validácie a regenerácie je aj v tomto prípade obmedzený maximálnym počtom iterácií, čím sa zabraňuje nekonečným cyklom a nestabilite generovania.

Výsledkom tejto fázy je konzistentný, validovaný a testovaný program, ktorý implementuje požadované správanie definované v prípade použitia.

2.4.6 Výstup systému a spracovanie spätnej väzby

Posledná fáza systému je zameraná na prezentáciu výsledného riešenia používateľovi a spracovanie jeho spätnej väzby. Výstup systému pozostáva zo zdrojového kódu, vizuál-

nej reprezentácie správania programu a interaktívneho rozhrania umožňujúceho ďalšiu iteráciu.

Generovanie diagramu aktivít Pre lepšie pochopenie správania programu je k výslednému kódu generovaný diagram aktivít. Diagram je vytváraný pomocou LLM na základe finálneho programu, pričom zdrojový kód predstavuje primárny zdroj pravdy.

Výsledkom je diagram v jazyku PlantUML, ktorý reprezentuje tok vykonávania programu na vyššej úrovni abstrakcie. Generovanie prebieha v dvoch krokoch – najprv je diagram vytvorený a následne podrobený revízii, ktorá identifikuje prípadné syntaktické alebo logické chyby.

Diagram je následne vykreslený a prezentovaný používateľovi ako vizuálna pomôcka na pochopenie generovaného riešenia.

Používateľské rozhranie Pre interakciu so systémom je implementované jednoduché grafické rozhranie pomocou knižnice Streamlit. Rozhranie má podobu chatového rozhrania doplneného o informačný panel, ktorý zobrazuje priebeh spracovania.

Hlavná časť rozhrania slúži na zadanie vstupného problému a postupné zobrazovanie výstupov systému vo forme správ. Používateľ zadáva zadanie v prirodzenom jazyku, ktoré je následne spracované systémom. Počas spracovania je vstupné pole dočasne deaktivované, čím sa signalizuje prebiehajúca generácia.

Na ľavej strane sa nachádza panel *System Status*, ktorý vizualizuje aktuálny stav jednotlivých fáz spracovania, konkrétne:

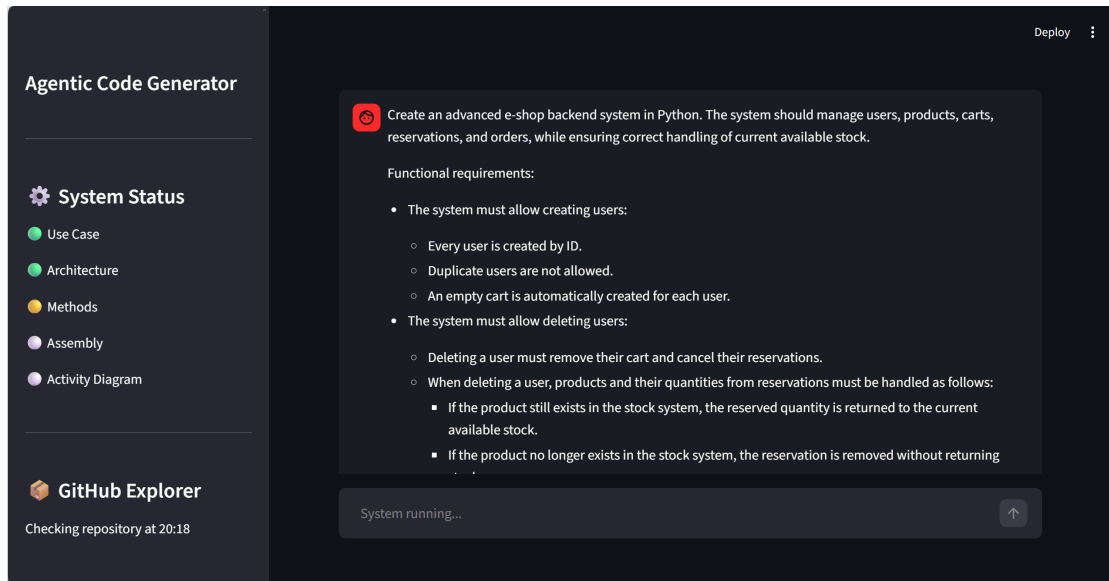
- generovanie prípadu použitia,
- návrh architektúry,
- generovanie metód,
- integráciu programu,
- generovanie diagramu aktivít.

Každá fáza je reprezentovaná indikátorom stavu (čakajúci-biela, prebiehajúci-žltá, dokončený-zelená), čo umožňuje používateľovi sledovať priebeh celého procesu v reálnom čase.

Súčasťou rozhrania je aj sekcia *GitHub Explorer*, ktorá zobrazuje priebežné logy o spracovaní externých repozitárov s časovým údajom. Táto sekcia poskytuje informácie o aktualizácii znalostnej bázy a prebiehajúcej analýze zdrojových kódov.

Po dokončení generovania systém zobrazí výsledný program spolu s diagramom aktivít a vyzve používateľa na zadanie spätnej väzby prostredníctvom vstupného poľa.

Ukážka používateľského rozhrania je znázornená na obr. 2.4.



Obr. 2.4: Ukážka používateľského rozhrania systému.

Rozhranie je navrhnuté ako jednoduché a funkčné, pričom jeho cieľom nie je poskytovať plnohodnotnú používateľskú aplikáciu, ale umožniť experimentálne overenie navrhovanej architektúry systému a interakciu s jednotlivými fázami spracovania.

Spracovanie spätnej väzby Po prezentovaní výsledného programu systém vyzve používateľa na zadanie spätnej väzby. Používateľ môže:

- potvrdiť správnosť riešenia,
- identifikovať chyby v implementácii,
- požadovať zmenu funkcionality.

Spätná väzba je následne analyzovaná pomocou LLM, ktorý ju klasifikuje do jednej z dvoch kategórií:

- **CODE_FIX** – ide iba o opravu implementácie existujúcich požiadaviek,
- **USE_CASE_UPDATE** – ide aj o zmenu alebo rozšírenie požiadaviek.

Klasifikácia je realizovaná porovnaním spätnej väzby s aktuálnym prípadom použitia a architektúrou systému.

Na základe tejto klasifikácie systém rozhoduje o ďalšom postupe:

- pri **CODE_FIX** sa proces vracia do fázy generovania metód,
- pri **USE_CASE_UPDATE** sa proces vracia do fázy generovania prípadu použitia.

Týmto spôsobom je implementovaný princíp *human-in-the-loop*, ktorý umožňuje iteratívne spresňovanie riešenia na základe používateľskej interakcie.

Ukladanie znalostí V prípade, že používateľ potvrdí správnosť riešenia, systém ponúkne možnosť uloženia vygenerovaných metód do znalostnej bázy. Tento proces zahŕňa transformáciu implementácie do ontologického modelu a jej uloženie do grafovej databázy. Takýmto selektívne riadeným ukladáním sa zabezpečuje kvalita a relevantnosť znalostnej bázy.

Výsledkom tejto fázy je nielen prezentácia riešenia, ale aj možnosť jeho iteratívneho zlepšovania a rozširovania znalostnej bázy systému, čím sa uzatvára celý cyklus navrhovaného agentického prístupu.

Kapitola 3

Evaluácia

Cieľom tejto kapitoly je experimentálne overiť navrhovaný prístup k agentickému generovaniu zdrojového kódu a porovnať jeho vlastnosti s vybranými existujúcimi riešeniami. Evaluácia je zameraná najmä na funkčnú správnosť generovaných riešení, kvalitu výsledného kódu a schopnosť systémov reagovať na spätnú väzbu a iteratívne zlepšovať svoje výstupy.

V nasledujúcich sekciách sú najskôr predstavené porovnávané systémy, následne testovacie zadania a metodika experimentu. Kapitola je zakončená prezentáciou a diskusiou dosiahnutých výsledkov.

3.1 Testovacie subjekty

V rámci evaluácie boli porovnávané štyri prístupy ku generovaniu zdrojového kódu, pričom okrem navrhovaného systému boli zvolené aj bežne dostupné a používané riešenia pre možné porovnanie:

- **Priame generovanie pomocou LLM** – riešenie generované modelom Mistral prostredníctvom oficiálneho webového rozhrania.
- **Navrhovaný agentický systém bez znalostnej bázy**
- **Navrhovaný agentický systém so znalostnou bázou** – rozšírenie predchádzajúceho prístupu o GraphRAG, ktorý poskytuje kontext vo forme relevantných implementácií.
- **Roo Code** – existujúci agentický systém umožňujúci výber jazykového modelu prostredníctvom API kľúča; v rámci experimentu bol použitý model Mistral.

Všetky prístupy využívajú rovnaký jazykový model, čo zabezpečuje férové porovnanie a umožňuje izolovať vplyv architektúry systému od kvality použitého modelu.

3.2 Testovacie zadania

V rámci evaluácie boli použité dve zadania s rôznou úrovňou komplexity. Zadania boli navrhnuté tak, aby pokrývali základné aj pokročilé aspekty návrhu softvérového systému v jazyku Python, vrátane práce so stavom, konzistencie dát a spracovania hraničných prípadov.

Rezervačný systém Prvé zadanie predstavuje stredne komplexný systém na správu rezervácií miestností. Systém umožňuje vytvárať a spravovať používateľov, miestnosti a rezervácie medzi nimi.

Zadanie kladie dôraz najmä na:

- správu vzťahov medzi entitami (používateľ, miestnosť, rezervácia),
- zabránenie konfliktov pri rezerváciách (napr. dvojité rezervácie),
- korektné spracovanie operácií ako vytváranie, rušenie a mazanie,
- udržiavanie konzistentného stavu systému pri rôznych sekvenciách operácií.

Súčasťou zadania sú aj explicitne definované pravidlá správania, napríklad ignorovanie neplatných operácií alebo automatické odstránenie súvisiacich rezervácií pri zmazaní používateľa alebo miestnosti.

E-shop backend Druhé zadanie predstavuje komplexnejší systém simulujúci backend e-shopu. Systém zahŕňa správu používateľov, produktov, košíkov, rezervácií a objednávok, pričom kladie dôraz na správne riadenie dostupného množstva produktov.

Zadanie rozširuje predchádzajúci problém o viaceré pokročilé aspekty:

- prácu s rezerváciami a ich vplyvom na dostupný sklad,
- spracovanie nákupného košíka a operácií nad ním,
- implementáciu procesu objednávky vrátane atomického spracovania,
- správu stavov objednávok a ich prechodov,
- riešenie konzistencie systému pri komplexných scenároch (napr. mazanie entít, neplatné operácie).

Zadanie zároveň obsahuje detailné pravidlá správania, ktoré definujú reakcie systému na rôzne hraničné situácie, ako napríklad neexistujúce entity, konflikty v dátach alebo zlyhanie operácií počas spracovania objednávky.

Obe zadania boli formulované tak, aby umožňovali jednoznačné testovanie správnosti riešenia a zároveň poskytovali dostatočnú variabilitu pre porovnanie rôznych prístupov ku generovaniu kódu.

3.3 Testovacia metodika

Cieľom testovania je objektívne porovnať správanie jednotlivých prístupov pri generovaní zdrojového kódu za rovnakých podmienok. Metodika je navrhnutá tak, aby zohľadňovala nedeterministický charakter jazykových modelov a zároveň umožňovala konzistentné vykonávanie testov.

Opakované generovanie Každé testovacie zadanie bolo pre každý porovnávaný prístup generované trikrát nezávisle. Tento prístup umožňuje znížiť vplyv náhodnosti pri generovaní výstupov a lepšie zachytiť stabilitu jednotlivých riešení.

V prípade, že vygenerované riešenie nevykazovalo správne správanie, bol systému poskytnutý jeden krok spätnej väzby, v ktorom bola stručne popísaná chyba. Následne bolo vykonané regenerovanie riešenia, čím sa overovala schopnosť systému reagovať na spätnú väzbu.

Testovanie pomocou scenárov Funkčná správnosť riešení je overovaná pomocou jednotkových testov, ktoré boli navrhnuté ako reprezentácia typických a hraničných situácií v systéme. Na základe týchto testov boli následne vytvorené formálne prípady použitia pre ich lepšiu reprezentáciu.

Testy pokrývajú:

- základné operácie systému,
- hraničné a chybové prípady,
- komplexné sekvencie operácií kombinujúce viaceré funkcionality.

Tento prístup umožňuje testovať nielen izolované operácie, ale aj celkové správanie systému v rôznych situáciách.

Na ilustráciu štruktúry testovacích scenárov je v ukážke 3.1 uvedený vybraný úryvok prípadu použitia v Cockburnovej notácii, ktorý reprezentuje komplexné správanie systému pri spracovaní objednávky.

```
Goal in Context: Process user orders while ensuring stock
                  consistency, atomicity and correct order creation.
```

```
Scope & Level: E-shop system, user-goal
```

```
Actors: Primary: User, Secondary: System
```

```
Preconditions: - User exists ...
```



```

Postconditions: - Successful checkout creates an order ...
Main Success Scenario: 1. Actor initiates checkout ...
Extensions:
Invalid conditions:
2a. User does not exist
-> Operation ignored ...
Atomicity: ...
Concurrency-like situations: ...
Post-deletion cases: ...
Complex flows: ...

```

Ukážka 3.1: Prípád použitia – checkout a spracovanie objednávky

Adaptačná vrstva Keďže jednotlivé generované riešenia sa môžu líšiť v názvoch tried, metód alebo v reprezentácii interného stavu, bol do testovacieho procesu zaradený adaptačný mechanizmus.

Adaptér zabezpečuje mapovanie rozhrania generovaného systému na jednotné testovacie rozhranie. Vďaka tomu je možné aplikovať rovnakú testovaciu sadu na všetky porovnávané prístupy bez ohľadu na konkrétnu implementáciu.

Navrhnutá metodika umožňuje systematické testovanie generovaných riešení v kontrolovanom prostredí a vytvára základ pre ich následné kvantitatívne vyhodnotenie.

3.4 Hodnotiace metriky

Na vyhodnotenie kvality generovaných riešení boli použité viaceré metriky, ktoré pokrývajú funkčnú správnosť aj kvalitu zdrojového kódu. Konkrétne ide o:

- úspešnosť testov vyjadrená v percentách,
- F1 skóre nad stavom systému,
- metriky kvality kódu (Pylint, Radon).

Prejdené testy Základnou metrikou je počet percent úspešne vykonaných testov. Každý test reprezentuje konkrétny scenár správania systému a je považovaný za úspešný, ak výsledný stav systému presne zodpovedá očakávanému stavu.

Táto metrika poskytuje binárnu informáciu o správnosti riešenia, avšak neberie do úvahy mieru čiastočne správnych stavov systému.

F1 skóre nad stavom systému Na detailnejšie hodnotenie správnosti bol použitý F1 ukazovateľ, ktorý umožňuje zachytiť aj čiastočnú zhodu medzi očakávaným a skutočným stavom systému.

Každý test porovnáva očakávaný a skutočný stav vo forme množín reprezentujúcich jednotlivé entity systému (napr. používatelia, produkty, rezervácie). Pre každú entitu sa vypočítajú hodnoty:

- **TP (True Positives)** – správne identifikované prvky,
- **FP (False Positives)** – prvky prítomné navyše,
- **FN (False Negatives)** – chýbajúce prvky.

Na základe týchto hodnôt sa počíta presnosť (precision) a úplnosť (recall):

$$\text{precision} = \frac{TP}{TP + FP} \quad (3.1)$$

$$\text{recall} = \frac{TP}{TP + FN} \quad (3.2)$$

Samotné F1 skóre je definované ako harmonický priemer presnosti a úplnosti:

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \quad (3.3)$$

Celkové skóre testu je následne vypočítané ako priemer F1 hodnôt cez všetky sledované entity systému.

Kvalita zdrojového kódu Okrem funkčnej správnosti bola hodnotená aj kvalita generovaného kódu pomocou nástrojov Pylint a Radon.

- **Pylint** poskytuje skóre kvality zdrojového kódu na škále 0–10 z pohľadu štýlu, čitateľnosti a dodržiavania odporúčaných programátorských praktík. Nástroj zároveň identifikuje potenciálne problémy, ako sú nevyužitú premennú, príliš dlhé riadky, redundantné konštrukcie alebo nevhodná štruktúra programu.
- **Radon** sa používa na analýzu komplexity kódu:
 - **cyklomatická komplexita (Cyclomatic complexity, CC)** meria zložitost' riadiacej logiky. Výstup má formu triedy (A–F) a číselnej hodnoty, napríklad B (10), kde:
 - * písmeno reprezentuje kategóriu komplexity,
 - * číslo udáva konkrétnu hodnotu cyklomatickej komplexity.

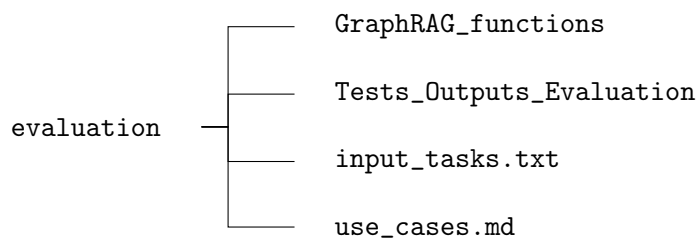
- **Index udržiavateľnosti (Maintainability Index, MI)** vyjadruje udržiavateľnosť kódu. Nástroj Radon poskytuje jeho kategorizovanú formu (A–C), kde trieda A reprezentuje vysokú udržiavateľnosť.

Tieto metriky dopĺňajú funkčné testovanie a umožňujú posúdiť aj kvalitu generovaného riešenia z pohľadu softvérového inžinierstva.

3.5 Elektronická príloha evaluácie

Súčasťou elektronickej prílohy je aj samostatná časť zameraná na experimentálnu evaluáciu navrhovaného systému. Táto časť repozitára obsahuje všetky podklady potrebné na reprodukovanie experimentov, testovanie generovaných riešení a analýzu výsledkov evaluácie.

Hlavná štruktúra evaluačnej časti repozitára je znázornená na obr. 3.1.



Obr. 3.1: Štruktúra evaluačnej časti repozitára

Adresár `GraphRAG_functions` obsahuje funkcie použité pri budovaní GraphRAG znalostnej bázy pri jednotlivých zadaniach.

Súbor `input_tasks.txt` obsahuje vstupné zadania použité počas experimentálnej evaluácie.

Súbor `use_cases.md` obsahuje testovacie scenáre vo forme prípadov použitia v Cockburnovej notácii.

Najrozsiahlejšou časťou evaluácie je adresár `Tests_Outputs_Evaluation`, ktorý obsahuje:

- automatizované testy použité pri evaluácii funkcionality generovaných riešení,
- validačné adaptéry zabezpečujúce jednotné testovanie jednotlivých systémov,
- logy zachytávajúce priebeh generovania a výsledky testovania,
- hodnotenia kvality zdrojového kódu,
- výstupy jednotlivých generovaní pre všetky testované systémy a scenáre.

Výstupy sú organizované podľa jednotlivých testovacích zadaní a evaluovaných systémov, pričom každý adresár obsahuje príslušné implementácie, logy, adaptéry a výsledky evaluácie.

3.6 Výsledky evaluácie

V tejto sekcii sú prezentované výsledky jednotlivých experimentov pre obe testovacie zadania. Výsledky sú rozdelené podľa jednotlivých testovacích zadaní a následne podľa porovnávaných prístupov.

3.6.1 Rezervačný systém

Priame generovanie pomocou Mistral

Pri prvom testovacom zadaní bolo použité priame promptovanie modelu Mistral prostredníctvom oficiálneho webového rozhrania. Zadanie bolo generované trikrát, pričom pri druhom pokuse sa v prvotne vygenerovanom riešení objavila syntaktická chyba spôsobená preklepom v dekorátore `@dataclass`. Po poskytnutí spätnej väzby model chybu opravil a následne vygeneroval korektné riešenie.

Všetky syntakticky validné riešenia úspešne prešli všetkými pripravenými testovacími scenármi a dosiahli priemerné F1 skóre 1.0.

Z pohľadu kvality kódu dosiahli všetky riešenia veľmi nízku cyklomatickú komplexitu. Priemerná komplexita podľa nástroja Radon bola vo všetkých prípadoch hodnotená stupňom A, pričom najkomplexnejšou metódou bola spravidla implementácia vytvárania rezervácií. Index udržiavateľnosti bol vo všetkých prípadoch taktiež hodnotený stupňom A.

Hodnotenie pomocou nástroja Pylint sa pohybovalo v intervale od 8.27/10 do 9.61/10. Najčastejšie problémy súviseli s:

- prekročením odporúčanej dĺžky riadkov,
- chýbajúcim novým riadkom na konci súboru,
- nepoužitými importmi,
- redefiníciou vstavaného identifikátora `id`,
- príliš veľkým počtom parametrov konštruktorov.

Súhrn nameraných hodnôt zobrazuje tabuľka 3.1.

Tabuľka 3.1: Výsledky LLM Mistral – rezervačný systém

Pokus	Testy	F1	Pylint	Radon CC	Radon MI
1	100 %	1.00	8.82	A (2.05)	A
2.1	syntaktická chyba	–	–	–	–
2.2	100 %	1.00	9.61	A (2.64)	A
3	100 %	1.00	8.27	A (2.57)	A
Priemer	100 %	1.00	8.90	A (2.42)	A

Navrhovaný agentický systém bez znalostnej bázy

Pri druhom testovanom prístupe bol použitý navrhovaný agentický systém bez využitia znalostnej bázy GraphRAG. Generovanie bolo realizované trikrát, pričom vo všetkých prípadoch systém úspešne vytvoril syntakticky validný program bez potreby dodatočnej spätnej väzby alebo regenerovania.

Všetky vygenerované riešenia úspešne prešli všetkými pripravenými testovacími scenármi a dosiahli priemerné F1 skóre 1.0.

Z pohľadu architektúry generovaných riešení bolo možné pozorovať mierne rozdiely medzi jednotlivými pokusmi. Niektoré implementácie využívali jednoduchšiu reprezentáciu rezervácií pomocou množín a tuple objektov, zatiaľ čo iné generovali detailnejšie dátové štruktúry vrátane explicitných identifikátorov rezervácií alebo evidencie zrušených rezervácií.

Priemerná komplexita podľa nástroja Radon sa pohybovala od A (3.33) po B (5.18), pričom najkomplexnejšími časťami boli najmä metódy `create_reservation` a operácie spracúvajúce rezervácie a konflikty časových intervalov.

Index udržiavateľnosti bol vo všetkých prípadoch hodnotený stupňom A, čo naznačuje dobrú udržiavateľnosť generovaného kódu aj napriek vyššej komplexite.

Hodnotenie pomocou nástroja Pylint sa pohybovalo v intervale od 8.47/10 do 9.48/10. Najčastejšie problémy súviseli s:

- prekročením odporúčanej dĺžky riadkov,
- chýbajúcim novým riadkom na konci súboru,
- nepoužitými premennými alebo importmi,
- nadmerným počtom návratových vetiev,
- redundantnými konštrukciami typu `else` po `return`.

Súhrn nameraných hodnôt zobrazuje tabuľka 3.2.

Tabuľka 3.2: Výsledky agentického systému bez GraphRAG – rezervačný systém

Pokus	Testy	F1	Pylint	Radon CC	Radon MI
1	100 %	1.00	9.39	A (3.33)	A
2	100 %	1.00	8.47	A (4.33)	A
3	100 %	1.00	9.48	B (5.18)	A
Priemer	100 %	1.00	9.11	A (4.28)	A

Agentický systém s GraphRAG

Pri treťom prístupe bol použitý navrhovaný agentický systém rozšírený o znalostnú bázu založenú na princípe GraphRAG. Počas generovania boli využívané aj relevantné implementačné príklady a štruktúrované znalosti získané zo znalostnej bázy.

Všetky tri generované riešenia úspešne prešli pripravenými testovacími scenármi a dosiahli priemerné F1 skóre 1.0. Generované implementácie sa však výraznejšie líšili vo vnútornej reprezentácii systému a v rozsahu implementovaných funkcionalít. Jednotlivé riešenia využívali odlišné dátové štruktúry, napríklad zoznamy, množiny alebo slovníky rezervácií, pričom niektoré implementácie obsahovali aj rozšírené funkcionality, ako napríklad filtrovanie systémového stavu alebo synchronizáciu pomocou zámkov.

Z pohľadu cyklomatickej komplexity dve riešenia dosiahli priemernú komplexitu stupňa B a jedno riešenie stupeň A. Najkomplexnejšími časťami systému boli metódy na vytváranie rezervácií a získavanie systémového stavu. Index udržiavateľnosti bol vo všetkých prípadoch hodnotený stupňom A.

Hodnotenie pomocou nástroja Pylint sa pohybovalo v intervale od 8.91/10 do 9.77/10. Najčastejšie problémy súviseli s:

- prekročením odporúčanej dĺžky riadkov,
- chýbajúcim novým riadkom na konci súboru,
- nepoužitými premennými.

Súhrn nameraných hodnôt zobrazuje tabuľka 3.3.

Tabuľka 3.3: Výsledky agentického systému s GraphRAG – rezervačný systém

Pokus	Testy	F1	Pylint	Radon CC	Radon MI
1	100 %	1.00	8.91	B (5.22)	A
2	100 %	1.00	9.26	A (3.64)	A
3	100 %	1.00	9.77	B (6.27)	A
Priemer	100 %	1.00	9.31	B (5.04)	A

Agentický systém Roo Code

Pri poslednom testovanom prístupe bol použitý agentický systém Roo Code integrovaný do prostredia Visual Studio Code. Generované riešenia boli výrazne rozsiahlejšie a formálnejšie štruktúrované v porovnaní s predchádzajúcimi prístupmi. Implementácie využívali objektovo orientovaný návrh s viacerými dátovými triedami reprezentujúcimi používateľov, miestnosti, rezervácie a časové intervaly.

Všetky tri riešenia úspešne prešli všetkými pripravenými testovacími scenármi a dosiahli priemerné F1 skóre 1.0. Implementácie obsahovali rozšírené validačné mechanizmy, pomocné interné metódy a viaceré štruktúry pre efektívne indexovanie rezervácií podľa používateľov alebo miestností.

Z pohľadu kvality kódu dosiahli všetky riešenia veľmi nízku cyklomatickú komplexitu. Priemerná komplexita podľa nástroja Radon bola vo všetkých prípadoch hodnotená stupňom A. Najkomplexnejšími časťami systému boli metódy na vytváranie rezervácií a kontrolu konfliktov časových intervalov. Index udržiavateľnosti bol vo všetkých prípadoch hodnotený stupňom A.

Hodnotenie pomocou nástroja Pylint sa pohybovalo v intervale od 5.17/10 do 8.26/10. Nižšie hodnotenie prvého riešenia bolo spôsobené predovšetkým veľkým množstvom formálnych problémov súvisiacich s formátovaním zdrojového kódu. Najčastejšie problémy zahŕňali:

- nadbytočné medzery na konci riadkov,
- prekročenie odporúčanej dĺžky riadkov,
- chýbajúci nový riadok na konci súboru,
- príliš veľký počet parametrov metód,
- nepoužité importy.

Súhrn nameraných hodnôt zobrazuje tabuľka 3.4.

Tabuľka 3.4: Výsledky agentického systému Roo Code – rezervačný systém

Pokus	Testy	F1	Pylint	Radon CC	Radon MI
1	100 %	1.00	5.17	A (2.12)	A
2	100 %	1.00	7.83	A (2.25)	A
3	100 %	1.00	8.26	A (2.46)	A
Priemer	100 %	1.00	7.09	A (2.28)	A

Súhrn výsledkov

Všetky testované prístupy boli schopné vygenerovať funkčné implementácie rezervačného systému. Syntakticky validné riešenia vo všetkých prípadoch úspešne prešli všetkými pripravenými testovacími scenármi a dosiahli priemerné F1 skóre 1.0. Výnimkou bolo jedno generovanie modelu Mistral, pri ktorom sa vyskytla syntaktická chyba spôsobená preklepom v dekorátore `@dataclass`, ktorá bola následne opravená pomocou spätnej väzby.

Z pohľadu kvality kódu dosiahli všetky riešenia priaznivé hodnotenia nástrojov Radon aj Pylint. Priemerná cyklomatická komplexita bola vo väčšine prípadov hodnotená stupňom A, pričom index udržiavateľnosti dosiahol vo všetkých analyzovaných riešeniach hodnotenie A. Rozdiely medzi jednotlivými prístupmi sa prejavili najmä v skóre nástroja Pylint a v rozsahu generovaného kódu.

Do súhrnu bola doplnená aj metrika vyjadrujúca počet generovaní, ktoré dosiahli 100 % úspešnosť testov, pričom v zátvorke je uvedený počet riešení dosiahnutých už na prvý pokus. Zatiaľ čo všetky prístupy dosiahli 100 % úspešnosť vo všetkých generovaniach, pri priamom generovaní pomocou modelu Mistral nebol tento výsledok v jednom prípade dosiahnutý na prvý pokus.

Súhrn priemerov nameraných hodnôt jednotlivých systémov zobrazuje tabuľka 3.5.

Tabuľka 3.5: Súhrn výsledkov – rezervačný systém

Prístup	Testy	F1	100% (1. pokus)	Pylint	Radon CC	Radon MI
Mistral	100 %	1.00	3/3 (2)	8.90	A (2.42)	A
Agentický systém	100 %	1.00	3/3 (3)	9.11	A (4.28)	A
GraphRAG	100 %	1.00	3/3 (3)	9.31	B (5.04)	A
Roo Code	100 %	1.00	3/3 (3)	7.09	A (2.28)	A

3.6.2 E-shop backend

Priame generovanie pomocou Mistral

Pri druhom testovacím zadaní bolo opätovne použité priame promptovanie modelu Mistral prostredníctvom oficiálneho webového rozhrania. Zadanie bolo generované trikrát, pričom pri každom pokuse bolo po prvotnom riešení poskytnuté ešte doplňujúce spätnoväzbové zadanie zamerané na odstránenie identifikovaných problémov.

Vo všetkých prípadoch boli vygenerované syntakticky validné riešenia. Testovanie však ukázalo, že implementácia komplexnejšieho e-shop backend systému predstavovala pre model náročnejšiu úlohu než predchádzajúci rezervačný systém. Viaceré riešenia nedokázali korektne spracovať všetky hraničné scenáre súvisiace najmä s manipuláciou

skladu a stavov objednávok. Napriek tomu väčšina testovacích scenárov dosahovala vysoké F1 skóre.

Z pohľadu kvality kódu dosiahli všetky riešenia nízku cyklomatickú komplexitu. Priemerná komplexita podľa nástroja Radon bola vo všetkých prípadoch hodnotená stupňom A, pričom najkomplexnejšími časťami implementácie boli najmä metódy `update_order_status`, `remove_from_cart` a `checkout`. Index udržiavateľnosti bol vo všetkých prípadoch hodnotený stupňom A.

Hodnotenie pomocou nástroja Pylint sa pohybovalo v intervale od 8.93/10 do 9.40/10. Najčastejšie problémy súviseli s:

- prekročením odporúčanej dĺžky riadkov,
- nepoužitými importmi,
- príliš hlbokým vnorením podmienok,
- príliš veľkým počtom booleovských výrazov v podmienkach,
- chýbajúcim novým riadkom na konci súboru.

Súhrn nameraných hodnôt zobrazuje tabuľka 3.6.

Tabuľka 3.6: Výsledky LLM Mistral – e-shop backend

Pokus	Testy	F1	Pylint	Radon CC	Radon MI
1.1	96 %	0.99	8.93	A (4.00)	A
1.2	100 %	1.00	9.06	A (4.13)	A
2.1	96 %	0.99	9.20	A (3.00)	A
2.2	100 %	1.00	9.40	A (3.10)	A
3.1	96 %	0.99	9.23	A (2.95)	A
3.2	100 %	1.00	9.25	A (3.05)	A
Priemer	98 %	0.995	9.18	A (3.37)	A

Agentický systém bez znalostnej bázy

Pri druhom testovacom zadaní bol evaluovaný navrhovaný agentický systém bez využitia znalostnej bázy. Zadanie bolo generované trikrát, pričom iba v jednom prípade bolo po neúspešnom prvotnom generovaní potrebné využitie aj opravného generovania na základe spätnej väzby.

Všetky generované riešenia boli syntakticky validné. Úspešnosť pri testovaní sa však medzi jednotlivými pokusmi líšila. Prvý a tretí pokus dosiahli plnú úspešnosť testovacích scenárov, zatiaľ čo druhý pokus dosiahol iba 80 % úspešnosť. Po aplikovaní spätnej väzby sa podarilo tento výsledok zlepšiť na 95 %.

Najčastejšie problémy sa vyskytovali pri zložitejších scenároch zahŕňajúcich:

- konzistenciu skladových zásob pri paralelných operáciách,
- správne spracovanie rezervácií medzi viacerými používateľmi,
- hraničné prípady pri checkout procese a jeho zlyhaní,
- komplexné sekvencie operácií kombinujúce viacero akcií nad systémom.

Z pohľadu kvality kódu dosiahli všetky riešenia veľmi vysoké hodnotenia nástroja Pylint (približne okolo hodnoty 9.5/10). Najčastejšie identifikované problémy súviseli najmä s:

- vyšším počtom vetvení v niektorých metódach,
- zachytávaním všeobecných výnimiek,
- prekročením odporúčanej dĺžky riadkov,
- nepoužitými premennými.

Cyklomatická komplexita bola vo všetkých prípadoch hodnotená stupňom B, pričom najkomplexnejšie časti implementácie predstavovali operácie práce s košíkom a checkout proces. Hodnoty komplexity sa pohybovali približne v intervale 5.2 až 7.1. Index udržiavateľnosti dosahoval vo všetkých riešeniach hodnotenie A. Súhrn namera- ných hodnôt zobrazuje tabuľka 3.7.

Tabuľka 3.7: Výsledky agentického systému bez GraphRAG – e-shop backend

Pokus	Testy	F1	Pylint	Radon CC	Radon MI
1	100 %	1.00	9.52	B (7.10)	A
2.1	80 %	0.92	9.5	B (6.36)	A
2.2	95 %	0.985	9.66	B (5.27)	A
3	100 %	1.00	9.67	B (6.00)	A
Priemer	94 %	0.98	9.59	B (6.18)	A

Agentický systém s GraphRAG

Pri druhom testovacom zadaní bol evaluovaný agentický systém rozšírený o GraphRAG znalostnú bázu. Zadanie bolo generované trikrát, pričom v dvoch prípadoch bolo po neúspešnom prvotnom generovaní využité aj opravné generovanie na základe spätnej väzby.

Výsledky ukazujú vysokú a zároveň stabilnejšiu úspešnosť oproti variantu bez znalostnej bázy. K plnej úspešnosti testovacích scenárov sme sa dostali pri všetkých 3 generovaniach. Priemerná úspešnosť tak dosiahla hodnotu 95 % pri priemernej hodnote F1 skóre 0.985.

Najslabší výsledok bol zaznamenaný pri prvom pokuse druhého generovania (86 %), kde sa objavili problémy najmä v oblastiach:

- aktualizácie skladových zásob pri opakovaných úpravách produktov,
- spracovania zmien cien produktov pri existujúcich rezerváciách,
- správneho obnovenia zásob pri rušení objednávok.

Na rozdiel od variantu bez znalostnej bázy sa však chyby nevyskytovali pri komplexných scenároch s viacerými používateľmi alebo pri základnej konzistencii rezervácií, čo naznačuje lepšie zachytenie doménovej logiky systému.

Hodnotenie pomocou nástroja Pylint sa pohybovalo v intervale od 9.40/10 do 9.89/10. Najčastejšie identifikované problémy zahŕňali:

- prekročenie odporúčanej dĺžky riadkov,
- vyšší počet vetvení a návratových príkazov,
- väčší počet atribútov tried,
- zachytávanie všeobecných výnimiek.

Cyklomatická komplexita bola vo väčšine prípadov hodnotená stupňom B, pričom v jednom prípade dosiahla stupeň A (hodnota 4.73), čo predstavuje najnižšiu komplexitu pri tomto systéme. Najkomplexnejšími časťami implementácie boli opäť operácie práce s košíkom a checkout proces. Index udržiavateľnosti bol vo všetkých riešeniach hodnotený stupňom A. Súhrn nameraných hodnôt zobrazuje tabuľka 3.8.

Tabuľka 3.8: Výsledky agentického systému s GraphRAG – e-shop backend

Pokus	Testy	F1	Pylint	Radon CC	Radon MI
1.1	91 %	0.98	9.74	B (5.55)	A
1.2	100 %	1.00	9.74	B (6.09)	A
2.1	86 %	0.945	9.89	A (4.73)	A
2.2	100 %	1.00	9.89	A (4.73)	A
3	100 %	1.00	9.40	B (6.36)	A
Priemer	95 %	0.985	9.73	B (5.49)	A

Agentický systém Roo Code

Posledným evaluovaným riešením bol agentický systém Roo Code. Zadanie bolo generované trikrát, pričom vo všetkých prípadoch bolo vykonané aj opravné generovanie po poskytnutí spätnej väzby z testovania.

Roo Code dosiahol priemernú úspešnosť testov 96 % a priemerné F1 skóre 0.99. Najlepšie výsledky dosiahli generovania 2.2 a 3.2, ktoré prešli všetkými testovacími scenármi. Najnižšia úspešnosť bola zaznamenaná pri pokuse 2.1 (90 %), pričom ostatné generovania dosiahli úspešnosť v intervale od 91 % do 100 %.

Z pohľadu kvality kódu dosiahol Roo Code najnižšie hodnotenia Pylint zo všetkých evaluovaných systémov, pričom skóre sa pohybovalo od 7.65/10 do 8.66/10. Väčšina identifikovaných problémov však nesúvisela priamo s funkcionalitou systému, ale najmä so štýlom a organizáciou kódu. Najčastejšie sa vyskytovali:

- koncové medzery,
- prekročenie odporúčanej dĺžky riadkov,
- nepoužívané importy,
- príliš malý počet verejných metód v triedach,
- nadbytočné vetvenia typu `else` po `return`,
- chýbajúci nový riadok na konci súboru,
- vyšší počet návratových príkazov alebo vetvení v niektorých metódach.

Napriek nižšiemu hodnoteniu Pylint dosiahol Roo Code najlepšie výsledky v oblasti cyklomatickej komplexity. Všetky generovania boli hodnotené stupňom A a priemerná komplexita sa pohybovala okolo hodnoty 2.67. Index udržiavateľnosti bol vo všetkých prípadoch hodnotený stupňom A.

Výsledné implementácie boli zároveň architektonicky najrozsiahlejšie zo všetkých testovaných riešení. Model často generoval väčší počet pomocných tried, enumov a abstrahovaných dátových štruktúr, čo viedlo k nižšej komplexite jednotlivých metód, ale zároveň k väčšiemu rozsahu kódu. Súhrn nameraných hodnôt zobrazuje tabuľka 3.9.

Tabuľka 3.9: Výsledky agentického systému Roo Code – e-shop backend

Pokus	Testy	F1	Pylint	Radon CC	Radon MI
1.1	91 %	0.98	7.93	A (2.68)	A
1.2	98 %	0.995	7.95	A (2.65)	A
2.1	90 %	0.98	8.21	A (2.61)	A
2.2	100 %	1.00	7.65	A (2.71)	A
3.1	98 %	0.99	8.56	A (2.69)	A
3.2	100 %	1.00	8.66	A (2.66)	A
Priemer	96 %	0.99	8.16	A (2.67)	A

Súhrn výsledkov

Všetky testované prístupy boli schopné vygenerovať syntakticky validné implementácie e-shop backend systému. Úspešnosť jednotlivých riešení sa však medzi prístupmi výraznejšie líšila než pri predchádzajúcom rezervačnom systéme. Najvyššiu priemernú úspešnosť testov dosiahlo priame generovanie pomocou modelu Mistral, zatiaľ čo agentické systémy vykazovali väčšiu variabilitu výsledkov medzi jednotlivými generovaniami.

Okrem priemernej úspešnosti bola analyzovaná aj schopnosť jednotlivých prístupov dosiahnuť plne korektné riešenie. Nami navrhovaný agentický systém aj Roo Code boli schopné vo väčšine prípadoch dosiahnuť 100 % úspešnosť testov, pričom tieto výsledky boli dosahované pri našom systéme aj na prvý pokus na rozdiel od iných systémov. Priame generovanie pomocou modelu Mistral a RooCode dosahovali 100 % úspešnosť až po spätnej väzbe.

Index udržiavateľnosti bol vo všetkých prípadoch hodnotený stupňom A. Rozdiely medzi jednotlivými prístupmi sa prejavili najmä v úspešnosti testov, cyklomatickej komplexite a výslednom skóre nástroja Pylint.

Súhrn priemerov nameraných hodnôt jednotlivých systémov spolu s počtom plne úspešných generovaní zobrazuje tabuľka 3.10.

Tabuľka 3.10: Súhrn výsledkov – e-shop backend

Prístup	Testy	F1	100% (1. pokus)	Pylint	Radon CC	Radon MI
Mistral	98 %	0.995	3/3 (0)	9.18	A (3.37)	A
Agentický systém	94 %	0.98	2/3 (2)	9.59	B (6.18)	A
GraphRAG	95 %	0.985	3/3 (1)	9.73	B (5.49)	A
Roo Code	96 %	0.99	2/3 (0)	8.16	A (2.67)	A

3.7 Diskusia výsledkov

Výsledky evaluácie ukázali, že všetky testované prístupy boli schopné generovať funkčné implementácie analyzovaných systémov. Výraznejšie rozdiely medzi jednotlivými riešeniami sa však prejavili najmä pri komplexnejšom e-shop backend systéme, ktorý obsahoval väčší počet stavových zmien, validačných pravidiel a závislostí medzi jednotlivými operáciami.

Pri jednoduchšom rezervačnom systéme dosahovali všetky prístupy konzistentne plne funkčné riešenia, zatiaľ čo pri e-shop systéme sa výraznejšie prejavili rozdiely v stabilite generovania, schopnosti riešiť hraničné scenáre a dosahovať plnú funkčnú správnosť už pri prvom pokuse.

Najproblematickejšími časťami implementácií boli najmä:

- manipulácia so skladovými zásobami,
- checkout proces,
- rezervácie produktov,
- validácia prechodov stavov objednávok.

3.7.1 Priame generovanie pomocou Mistral

Priame generovanie pomocou modelu Mistral dosiahlo najvyššiu priemernú úspešnosť testov spomedzi všetkých evaluovaných prístupov. Výsledky zároveň pôsobili relatívne stabilne, keďže jednotlivé generovania vykazovali podobnú mieru úspešnosti a po poskytnutí spätnej väzby model opakovane dokázal opraviť identifikované problémy.

Zaujímavým zistením však bolo, že pri komplexnejšom e-shop backend systéme model nedokázal ani raz vygenerovať plne korektné riešenie už na prvý pokus. To sa prejavilo aj v metrike 100 % úspešnosti, kde boli plne funkčné riešenia dosahované až po aplikovaní spätnej väzby. Naznačuje to, že aj pri veľmi kvalitných generatívnych modeloch môže byť pri komplexnejších stavových systémoch potrebná iteratívna interakcia a postupné spresňovanie riešenia.

Prekvapujúcim výsledkom bola aj syntaktická chyba pri generovaní rezervačného systému, kde model vytvoril nevalidný dekorátor `@dataclass`. Veľké jazykové modely sú pritom vo všeobecnosti považované za veľmi silné pri generovaní syntakticky korektného kódu. Hoci išlo len o jednoduchý preklep, výsledok ukazuje, že ani pri relatívne jednoduchých úlohách nemožno úplne predpokladať bezchybnú syntaktickú správnosť generovaného riešenia.

Z pohľadu kvality kódu dosahoval Mistral:

- stabilne vysoké hodnotenia nástroja Pylint,

- nízku cyklomatickú komplexitu,
- konzistentne vysoký index udržiavateľnosti.

3.7.2 Navrhovaný agentický systém bez znalostnej bázy

Navrhovaný agentický systém bez využitia GraphRAG dosiahol pri rezervačnom systéme veľmi dobré výsledky a vo všetkých prípadoch dokázal vygenerovať plne funkčné riešenia už na prvý pokus bez potreby spätnej väzby. Táto vlastnosť sa prejavila aj v metrike 100 % úspešnosti, kde všetky generovania dosiahli plnú funkčnú správnosť už pri prvom pokuse.

Pri komplexnejšom e-shop backend systéme bol systém taktiež schopný až v dvoch prípadoch dosiahnuť plne korektné riešenie už pri prvom generovaní, avšak tieto výsledky neboli konzistentné naprieč všetkými pokusmi.

Výsledky zároveň ukázali nedeterministickosť agentického prístupu. Niektoré generovania dosiahli takmer perfektné výsledky, zatiaľ čo iné zlyhávali v časti testovacích scenárov.

Výsledky preto naznačujú, že agentický systém je schopný produkovať veľmi kvalitné riešenia vrátane plne funkčných implementácií, avšak ich kvalita zostáva závislá od konkrétneho priebehu generovania.

Zaujímavým pozorovaním bolo aj veľmi vysoké hodnotenie nástroja Pylint. Agentický systém bez znalostnej bázy dosahoval veľmi dobré priemerné hodnotenia zo všetkých testovaných riešení, a to aj v prípadoch, keď funkcionálna implementácia nebola úplne správna.

Naopak, cyklomatická komplexita generovaných riešení bola pri tomto prístupe pomerne vysoká. Agentický systém často generoval rozsiahle metódy obsahujúce veľké množstvo podmienok a vetvení, najmä pri implementácii práce so skladovými zásobami, rezerváciami alebo objednávkami.

3.7.3 Agentický systém s GraphRAG

Rozšírenie agentického systému o GraphRAG znalostnú bázu prinieslo zlepšenie stability výsledkov pri komplexnejšom e-shop backend systéme. V porovnaní s verziou bez znalostnej bázy dosahoval systém:

- vyššiu priemernú úspešnosť testov,
- vyšší počet plne úspešných generovaní,
- stabilnejšie výsledky medzi jednotlivými pokusmi.

Výsledky naznačujú, že využitie implementačných príkladov a štruktúrovaných znalostí pomohlo modelu lepšie zachytiť požadované návrhové vzory a validačné pravidlá.

Zlepšenie sa prejavilo aj v metrike 100 % úspešnosti, kde systém dosahoval plne funkčné riešenia častejšie než variant bez znalostnej bázy.

Aj pri GraphRAG prístupe však zostala prítomná určitá miera nedeterministickosti. Jedno generovanie dosahovalo plne korektné výsledky už po prvom pokuse, zatiaľ čo ostatné riešenia obsahovali logické chyby a bola potrebná spätná väzba. Znalostná báza teda síce zlepšovala stabilitu systému, ale úplne neeliminovala variabilitu generovania typickú pre agentické prístupy.

Z pohľadu kvality kódu dosahoval systém:

- veľmi vysoké hodnotenia nástroja Pylint,
- stabilný index udržiavateľnosti,
- mierne vyššiu cyklomatickú komplexitu.

Vyššie hodnoty Radon CC boli pravdepodobne spôsobené tým, že systém často implementoval väčšiu časť logiky priamo v jednotlivých metódach namiesto jej rozdelenia do menších abstrahovaných komponentov.

3.7.4 Agentický systém Roo Code

Roo Code predstavoval odlišný typ agentického prístupu než navrhovaný systém. Generované implementácie boli výrazne rozsiahlejšie, viac abstrahované a architektonicky komplexnejšie. Roo Code často vytváral:

- väčší počet pomocných tried,
- enumy,
- interné dátové štruktúry,
- pomocné validačné mechanizmy.

Tento prístup viedol k veľmi nízkej cyklomatickej komplexite jednotlivých metód. Výsledné hodnoty Radon CC boli najlepšie zo všetkých testovaných prístupov.

Na druhej strane Roo Code dosahoval výrazne nižšie hodnotenia nástroja Pylint. Väčšina problémov súvisela najmä s formátovaním a organizáciou zdrojového kódu. Často sa objavovali:

- koncové medzery,
- nepoužívané importy,

- príliš dlhé riadky,
- redundantné vetvenia,
- nadmerný počet návratových príkazov.

Z pohľadu funkčnej správnosti bol Roo Code schopný dosiahnuť plne korektné riešenia, čo sa prejavilo aj v metrike 100 % úspešnosti. Tieto výsledky však neboli dosahované na prvý pokus a vykazovali určitú mieru variability medzi jednotlivými generovaniami.

3.7.5 Súhrn diskusie

Na základe výsledkov evaluácie bolo možné identifikovať viaceré charakteristické vlastnosti jednotlivých prístupov. Každé riešenie vykazovalo odlišné silné a slabé stránky z pohľadu funkcionality, stability generovania aj kvality výsledného kódu.

Zásadným zistením je rozdiel medzi priemernou úspešnosťou generovania a schopnosťou dosiahnuť plne korektné riešenie. Priame generovanie pomocou modelu Mistral a RooCode dosahovali najvyššiu priemernú úspešnosť, avšak nedokázali generovať plne funkčné riešenia už na prvý pokus pri komplexnejšom e-shop backend systéme.

Naopak, navrhovaný agentický systém bol schopný v polovici prípadoch dosiahnuť 100 % úspešnosť už pri prvom generovaní, čo predstavuje významnú výhodu z pohľadu efektivity generovania. Nižšia priemerná úspešnosť bola spôsobená vyššou mierou nedeterministickosti, ktorá viedla k väčšej variabilite medzi jednotlivými pokusmi.

Rozšírenie systému o GraphRAG znalostnú bázu prinieslo zlepšenie stability a zvýšenie počtu plne úspešných generovaní, čím sa priblížilo vlastnostiam stabilnejších prístupov bez výraznej straty schopnosti generovať kvalitné riešenia.

Z pohľadu kvality kódu dosahoval navrhovaný agentický systém veľmi vysoké hodnotenia nástroja Pylint, avšak za cenu vyššej cyklomatickej komplexity. Tento kompromis naznačuje, že systém uprednostňuje explicitné riešenie logiky pred jej abstrahovaním.

Prehľad identifikovaných výhod a nevýhod jednotlivých prístupov je uvedený v tabuľke 3.11. Uvedené zistenia vychádzajú z konkrétnych experimentálnych scenárov použitých v tejto práci a ich platnosť môže byť ovplyvnená charakterom testovaných úloh.

Tabuľka 3.11: Súhrn výhod a nevýhod jednotlivých prístupov

Prístup	Výhody	Nevýhody
Mistral	• najvyššia priemerná úspešnosť testov, • stabilné výsledky medzi generovaniami, • nízka cyklomatická komplexita, • dobré hodnotenia nástroja Pylint.	• nemožnosť dosiahnuť 100 % riešenie na prvý pokus pri komplexných úlohách, • výskyt syntaktickej chyby, • menšia modularita riešení.
Agentický systém bez GraphRAG	• schopnosť dosiahnuť 100 % riešenie už na prvý pokus, • veľmi vysoké hodnotenia nástroja Pylint.	• prítomná nedeterministickosť, • najvyššia cyklomatická komplexita, • nižšia priemerná úspešnosť testov.
Agentický systém s GraphRAG	• vyššia priemerná úspešnosť testov než bez znalostnej bázy, • vyšší počet plne úspešných generovaní, • stabilnejšie výsledky medzi pokusmi, • najvyššie hodnotenia nástroja Pylint.	• pretrvávajúca nedeterministickosť, • vyššia cyklomatická komplexita.
Roo Code	• veľmi dobrá priemerná úspešnosť testov, • najnižšia cyklomatická komplexita, • vysoká modularita implementácií.	• schopnosť dosiahnuť 100 % riešenie už na prvý pokus, • najnižšie hodnotenia nástroja Pylint, • vysoká variabilita výsledkov.

3.7.6 Vyhodnotenie výskumných otázok a hypotéz

Na základe výsledkov evaluácie bolo možné vyhodnotiť formulované výskumné otázky a hypotézy.

Hypotéza **RH1.1** bola čiastočne potvrdená. Agentické systémy síce v priemere nedosiahli vyššiu úspešnosť testovacích scenárov ani vyššie F1 skóre než priame promptovanie modelu Mistral, avšak nami navrhovaný systém v niektorých prípadoch dokázal vygenerovať plne funkčné riešenia už pri prvom generovaní. Výsledky zároveň ukázali vyššiu mieru nedeterministickosti agentických prístupov, ktorá spôsobovala variabilitu medzi jednotlivými generovaniami.

Hypotéza **RH1.2** bola čiastočne potvrdená. Navrhovaný agentický systém dosahoval lepšie hodnotenia nástroja Pylint než priame promptovanie, avšak zároveň vykazoval vyššiu cyklomatickú komplexitu podľa metriky Radon CC. Naopak, agentický systém Roo Code dosahoval nižšiu cyklomatickú komplexitu, avšak horšie hodnotenia nástroja Pylint.

Hypotéza **RH2** bola potvrdená. Rozšírenie navrhovaného agentického systému o GraphRAG znalostnú bázu viedlo k zvýšeniu úspešnosti testovacích scenárov, vyššiemu F1 skóre a zároveň k stabilnejším výsledkom generovania.

Hypotéza **RH3.1** bola čiastočne potvrdená. Navrhovaný agentický systém založený na fragmentácii úloh nedosiahol v priemere vyššiu funkčnú správnosť než porovnávaný agentický systém Roo Code. Avšak vykazoval schopnosť vyššieho F1 skóre a vyššej funkčnej správnosti pri prvotnom generovaní.

Hypotéza **RH3.2** bola čiastočne potvrdená. Navrhovaný agentický systém dosahoval veľmi vysoké hodnotenia kvality kódu podľa nástroja Pylint, avšak v oblasti cyklomatickej komplexity nedosiahol lepšie výsledky než Roo Code, ktorý generoval architektonicky viac rozdelené riešenia.

Záver

Zhrnutie

Bakalárska práca sa zaoberala využitím umelej inteligencie pri vývoji softvéru, konkrétne návrhom a implementáciou agentického systému na generovanie zdrojového kódu z prirodzeného jazykového zadania.

V analytickej časti práce boli spracované poznatky z oblasti generovania zdrojového kódu pomocou veľkých jazykových modelov, UI-asistovaného vývoja softvéru, agentických systémov, GraphRAG prístupov a reprezentácie znalostí pomocou ontológií. Súčasťou analýzy bolo aj porovnanie existujúcich AI nástrojov a agentických systémov používaných v softvérovom inžinierstve.

Na základe vykonanej analýzy boli formulované výskumné otázky a hypotézy zamerané na overenie prínosu agentických systémov, fragmentácie generovania a využitia GraphRAG znalostnej bázy z pohľadu funkčnej správnosti a kvality generovaného zdrojového kódu.

Praktická časť práce pozostávala z návrhu a implementácie vlastného agentického systému realizovaného pomocou frameworku LangGraph. Navrhované riešenie využíva viacfázový pracovný postup, v ktorom sa vstupné zadanie postupne transformuje do štruktúrovanej reprezentácie prípadu použitia, následne do návrhu architektúry a nakoniec do implementácie jednotlivých metód programu. Generovanie je podporené automaticky aktualizovanou GraphRAG znalostnou bázou využívajúcou grafovú databázu Neo4j s ontológiou na organizáciu znalostí. Významnou súčasťou systému je aj iteratívna validácia generovaných výstupov, spätná väzba používateľa a mechanizmus regenerovania chybných častí riešenia.

Súčasťou práce bola aj experimentálna evaluácia navrhovaného systému. Výsledky boli porovnávané s priamym generovaním pomocou modelu Mistral a s existujúcim agentickým systémom Roo Code. Evaluácia ukázala, že navrhovaný agentický prístup je schopný v niektorých prípadoch generovať plne funkčné riešenia už na prvý pokus a dosahovať veľmi vysokú kvalitu kódu podľa metrík Pylint. Zároveň sa však potvrdilo, že agentické systémy vykazujú vyššiu mieru nedeterministickosti než priame generovanie pomocou jednej LLM, čo sa prejavilo aj na priemernej úspešnosti navrhovaného riešenia. Výsledky zároveň naznačili, že využitie GraphRAG znalostnej bázy prispelo

k vyššej stabilite výsledkov a zlepšeniu funkčnej správnosti.

Výsledky naznačujú, že kombinácia fragmentácie problému, štruktúrovaných znalostí a iteratívnej validácie predstavuje perspektívny smer ďalšieho vývoja UI-asistovaného softvérového inžinierstva.

Hlavné prínosy práce

Hlavné prínosy tejto práce možno zhrnúť do nasledujúcich bodov:

- návrh vlastného agentického systému založeného na fragmentácii generovania, iteratívnej validácii a spätnej väzbe používateľa,
- implementácia viacagentovej architektúry využívajúcej framework LangGraph s automaticky aktualizovanou znalostnou bázou využívajúcou grafovú databázu Neo4j,
- experimentálne porovnanie navrhovaného systému s priamym generovaním pomocou LLM a existujúcim agentickým systémom Roo Code,
- identifikácia silných a slabých stránok navrhovaného prístupu v porovnaní s ostatnými riešeniami z pohľadu funkčnej správnosti, stability generovania a kvality zdrojového kódu.

Prínos práce možno vnímať aj z osobného a odborného hľadiska. Práca zároveň prispela k rozšíreniu praktických skúseností autora v oblasti veľkých jazykových modelov, agentických systémov, grafových databáz, frameworku LangGraph a moderných prístupov UI-asistovaného vývoja softvéru. Zároveň práca poskytla skúsenosti s návrhom a implementáciou rozsiahlejšieho softvérového systému pozostávajúceho z viacerých spolupracujúcich komponentov.

Možné smery vývoja

Navrhovaný systém predstavuje experimentálny agentický prístup ku generovaniu zdrojového kódu, ktorý je možné v budúcnosti ďalej rozširovať a zdokonaľovať.

Jedným z možných smerov ďalšieho vývoja je rozšírenie systému o podporu ďalších programovacích jazykov a typov softvérových projektov. Súčasná implementácia je zameraná primárne na generovanie Python aplikácií, pričom rozšírenie o ďalšie jazyky by si vyžadovalo úpravu znalostnej bázy, validačných mechanizmov aj promptovacích stratégií.

Zaujímavým smerom ďalšieho rozvoja môže byť aj rozšírenie architektonického agenta o podporu známych architektonických štýlov a návrhových princípov zo softvérového inžinierstva. Systém by mohol automaticky rozpoznávať vhodný architektonický štýl podľa charakteru zadania.

Priestor na zlepšenie existuje aj v oblasti stability a deterministickosti agentických systémov. Budúci výskum by sa mohol zamerať na efektívnejšie stratégie koordinácie agentov s cieľom znížiť variabilitu výsledkov medzi jednotlivými generovaniami.

Ďalším možným smerom výskumu je realizácia rozsiahlejšej experimentálnej evaluácie zameranej na porovnanie viacerých agentických systémov, rôznych veľkých jazykových modelov a širšieho spektra testovacích scenárov.

Možným rozšírením systému je aj vytvorenie pokročilejšieho používateľského rozhrania, ktoré by umožňovalo jednoduchšiu interakciu so systémom a jeho praktické nasadenie.

Literatúra

- [1] Felipe Almeida and Geraldo Xexéo. Word embeddings: A survey. *arXiv preprint arXiv:1901.09069*, 2019.
- [2] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. Small language models are the future of agentic ai, 2025.
- [3] Alistair Cockburn and Lord Cockburn. *Writing effective use cases*. Pearson Education India, 2008.
- [4] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Muling Wu, Yunbo Tao, Ming Zhang, Mingxu Chai, Jessica Fan, Zhiheng Xi, Rui Zheng, Yueming Wu, Ming Wen, Tao Gui, Qi Zhang, Xipeng Qiu, and Xuanjing Huang. What is wrong with your code generated by large language models? an extensive study. *Science China Information Sciences*, 69(1):112107, January 2026.
- [5] Mohamed Amine Ferrag, Norbert Tihanyi, and Merouane Debbah. From llm reasoning to autonomous ai agents: A comprehensive review, 2025.
- [6] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. Security weaknesses of copilot-generated code in github projects: An empirical study. *ACM Trans. Softw. Eng. Methodol.*, 34(8), October 2025.
- [7] Thomas R Gruber. Toward principles for the design of ontologies used for knowledge sharing? *International journal of human-computer studies*, 43(5-6):907–928, 1995.
- [8] Xiaodong Gu, Meng Chen, Yalan Lin, Yuhan Hu, Hongyu Zhang, Chengcheng Wan, Zhao Wei, Yong Xu, and Juhong Wang. On the effectiveness of large language models in domain-specific code generation. *ACM Trans. Softw. Eng. Methodol.*, 34(3), February 2025.
- [9] Junda He, Christoph Treude, and David Lo. Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. *ACM Trans. Softw. Eng. Methodol.*, 34(5), May 2025.

- [10] Nam Huynh and Beiyu Lin. Large language models for code generation: A comprehensive survey of challenges, techniques, evaluation, and applications, 2025.
- [11] Ivar Jacobson. *Object-oriented software engineering: a use case driven approach*. Pearson Education India, 1993.
- [12] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, and Ji-Rong Wen. KG-agent: An efficient autonomous agent framework for complex reasoning over knowledge graph. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9505–9523, Vienna, Austria, July 2025. Association for Computational Linguistics.
- [13] Marián Kica. Vývoj softvéru kombinujúci interné a externé zdroje, 2025. Diplomová práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyziky a informatiky.
- [14] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering, 2025.
- [15] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. Structured chain-of-thought prompting for code generation. *ACM Trans. Softw. Eng. Methodol.*, 34(2), January 2025.
- [16] Jia Li, Chongyang Tao, Jia Li, Ge Li, Zhi Jin, Huangzhao Zhang, Zheng Fang, and Fang Liu. Large language model-aware in-context learning for code generation. *ACM Trans. Softw. Eng. Methodol.*, 34(7), August 2025.
- [17] Shuang Li, Yuntao Cheng, Jinfu Chen, Jifeng Xuan, Sen He, and Weiyi Shang. Assessing the performance of ai-generated code: A case study on github copilot. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, pages 216–227, 2024.
- [18] Yue Liu, Thanh Le-Cong, Ratnadira Widiasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D. Le, and David Lo. Refining chatgpt-generated code: Characterizing and mitigating code quality issues. *ACM Trans. Softw. Eng. Methodol.*, 33(5), June 2024.
- [19] Zhijie Liu, Yutian Tang, Xiapu Luo, Yuming Zhou, and Liang Feng Zhang. No need to lift a finger anymore? assessing the quality of code generation by chatgpt. *IEEE Transactions on Software Engineering*, 50(6):1548–1584, 2024.

- [20] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. An empirical study of the non-determinism of chatgpt in code generation. *ACM Trans. Softw. Eng. Methodol.*, 34(2), January 2025.
- [21] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph retrieval-augmented generation: A survey. *ACM Trans. Inf. Syst.*, 44(2), December 2025.
- [22] Ranjan Sapkota, Konstantinos I. Roumeliotis, and Manoj Karkee. Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges. *Information Fusion*, 126:103599, February 2026.
- [23] Florian Tambon, Arghavan Moradi-Dakhel, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Giuliano Antoniol. Bugs in large language models generated code: an empirical study. *Empirical Software Engineering*, 30(3):65, February 2025.
- [24] Jahonoro Tojjeva. Software system test generation, 2025. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyziky a informatiky.
- [25] Catherine Tony, Nicolás E. Díaz Ferreyra, Markus Mutas, Salem Dhif, and Riccardo Scandariato. Prompting techniques for secure code generation: A systematic investigation. *ACM Trans. Softw. Eng. Methodol.*, 34(8), October 2025.
- [26] Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari Sahraoui. Exploring parameter-efficient fine-tuning techniques for code generation with large language models. *ACM Trans. Softw. Eng. Methodol.*, 34(7), August 2025.
- [27] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Hao Chen, Yilin Xiao, Chuang Zhou, Junnan Dong, Yi Chang, and Xiao Huang. A survey of graph retrieval-augmented generation for customized large language models, 2025.

Príloha A: obsah elektronickej prílohy

Súčasťou elektronickej prílohy je zdrojový kód implementovaného systému spolu s podkladmi a výstupmi použitými pri experimentálnej evaluácii, vrátane testovacích scenárov, validačných adaptérov, logov a výstupov jednotlivých generovaní. Elektronická príloha je dostupná vo verejnom GitHub repozitári na adrese <https://github.com/SamSok31/BakalarskaPraca>.